

Autonomous Information Seeking: A Roadmap for Agentic Recommender Systems

Xinyu Lin¹ Yashar Deldjoo²✉ Sunhao Dai³ Honghui Bao¹ Xiaopeng Ye³ Fatemeh Nazary² Wenjie Wang⁴ Tommaso Di Noia² Jun Xu³ Tat-Seng Chua¹

¹National University of Singapore ²Polytechnic University of Bari

³Renmin University of China ⁴University of Science and Technology of China

xylin1028@gmail.com, deldjoo@acm.org

The rapid integration of large language model-based agents into recommender systems has driven a shift from static, ranking-based pipelines toward autonomous and interactive systems that can reason, plan, and act. This survey provides a comprehensive overview of this emerging landscape by introducing a unified taxonomy grounded in the level of autonomy and three core paradigms of agentic recommender systems: agent-assisted recommendation, agent-as-recommender, and agent-as-user-simulator. The autonomy framework organizes existing methods along increasing capabilities in proactivity, context awareness, interaction flexibility, and adaptivity. Building on this framework, the survey analyzes how each paradigm adopts different agentic architectures and how agents enhance key components such as profiles, memory, tool use, workflows, and optimization mechanisms. We further examine evaluation methodologies for agentic recommendation, covering automated metrics, LLM-based judging, and simulation-based assessment, and discuss their limitations in capturing reasoning quality, user experience, and system behavior. Beyond existing evaluation protocols, we further discuss unresolved issues in evaluating agentic recommender systems, including trajectory-level assessment, agent contribution analysis, and calibration of user simulation. Lastly, the survey outlines open challenges in lifelong user modeling, contextual abstraction, multimodal alignment, controllability, trustworthiness, privacy, scalability, and efficiency. Together, these analyses establish a unified foundation for understanding the current progress of agentic recommender systems and highlight promising opportunities for developing more autonomous, reliable, and human-aligned recommendation agents.

Keywords: Agentic Recommender System, Level of Autonomy, LLM Agent, Agent-assisted Recommender, Agent as Recommender, Agent as User Simulator

✉ Corresponding author.

1. Introduction

Recommender systems (RS) have traditionally been evaluated by how accurately they predict user preferences and rank items based on historical user–item interactions. Classical approaches—from collaborative filtering to modern deep retrieval and ranking architectures trained on clicks, ratings, and purchases—encode past behavior into latent representations and, given a user and context, output a ranked list of candidate items. This paradigm is highly effective for the dominant interaction pattern in today’s platforms: the system curates options and the user chooses among them (*e.g.*, selecting a movie from a ranked homepage list).

Table 1 | Modernized comparison of recommender system paradigms.

Dimension	Classical RS	LLM-based RS (generative / prompt-based)	Agentic RS
Recommendation Goal	Personalized ranking: match past user–item interactions via shallow user models.	Personalized ranking & language generation: infer preferences via prompts and generate textual outputs.	<i>Goal-oriented decision support</i> : produce recommendations under constraints via multi-step reasoning and actions.
Proactivity	Reactive: suggest only when RS is explicitly requested.	Mostly reactive: respond to prompts and conversational requests.	<i>Mixed-initiative potential</i> : be able to ask clarifying questions, propose plans, and surface trade-offs.
Context Awareness	Limited to behavior logs and static features.	Uses context within the LLM’s window with optional retrieved snippets.	<i>Situational & tool-grounded</i> : integrates logs, profiles, and external data sources (e.g., web/APIs).
Interactivity	Single-step recommendation or static re-ranking.	Natural-language interaction, including single- and multi-turn interactions, but typically without explicit planning/control.	<i>Multi-step workflows</i> : plan/act/verify cycles with error recovery and iterative refinement.
Adaptivity	Offline training with periodic model updates.	Limited in-session adaptation (e.g., in-context learning).	<i>In-session and lifelong adaptation</i> : profile/memory updates via reflection or feedback-driven adjustment.
Memory	No explicit working memory beyond logs and engineered features.	Mostly short context window in single-turn recommendation; conversation history in multi-turn recommendation.	<i>Structured memory</i> : working memory with optional episodic/semantic stores across steps/sessions.
Tools & Knowledge	Fixed catalog/indices without access to external knowledge.	Parametric internal knowledge and optional external knowledge via retrieval.	<i>External tool use</i> : invokes retrieval, rankers, search, APIs, or constraints solvers for grounded actions.
Collaboration	Independent optimization of models with little to no interaction between distinct modules.	Integrate LLMs with conventional models via pre-defined sequential workflow.	<i>Inter-agent collaboration</i> : enables dynamic collaboration between specialized agents to solve complex tasks through negotiation and feedback.

However, emerging recommendation scenarios increasingly involve **complex, multi-step user goals** that are difficult to support with a single “one-shot” ranked list, even when the underlying ranker is highly accurate. Users may want help planning a week-long vacation under constraints (budget, timing, transportation, lodging), adapting plans as external factors change (weather, delays), or coordinating a wardrobe under style, budget, and occasion constraints. What makes these scenarios challenging is not merely preference prediction, but **constraint reasoning, iterative refinement, and decision support** under incomplete and evolving information. Importantly, many real systems—and many works surveyed in this paper—still ultimately provide a ranked list or a shortlist of options. The key shift is that the recommendation is increasingly produced through a **multi-step workflow** that must interpret goals, elicit constraints, consult tools or knowledge sources, and revise outputs as new information arrives.

The rapid progress of foundation models (FMs) and large language models (LLMs) has opened new avenues for recommendation through natural-language interfaces and stronger semantic reasoning. A growing body of work surveys LLM-enhanced recommendation and generative recommendation, emphasizing how LLMs can act as encoders, re-rankers, conversational interfaces, or generators of recommendations and explanations (e.g., Li et al. (2024); Wu et al. (2024b); Zhao et al. (2024b)), and the broader Gen-RecSys view in Deldjoo et al. (2024a,b)). Yet, **LLM-based RS are not necessarily agentic**. Many LLM-based systems remain predominantly reactive: they respond to prompts and in-context history but do not explicitly plan, call external tools, maintain persistent memory, or coordinate specialized modules beyond a single forward pass. This matters because recommendation is a system problem: user modeling, retrieval, ranking, constraint handling, and evaluation often require structured control over external components and environment interactions. The comparisons between classical RS, LLM-based RS, and Agentic RS are summarized in Table 1, from seven dimensions, including recommendation goal, proactivity, context awareness, interactivity, adaptivity, memory, and tools & knowledge.

From “models that rank” to “systems that pursue goals”. This survey focuses on *agentic recommender systems* (ARS): systems that integrate one or more agents—often instantiated by LLMs—with explicit mechanisms for *action selection* (e.g., tool use and memory updates) inside a recommendation environment. Concretely, an agent in ARS can (i) interpret a natural-language goal, (ii) decompose it into sub-tasks, (iii) invoke tools such as retrieval, ranking, filtering, web search, or APIs, (iv) track intermediate state (e.g., constraints, candidate sets, user feedback), and (v) refine recommendations iteratively. This level of interactivity and controllable action *enables* a shift from “models that rank” toward “systems that pursue goals,” though we stress that today’s systems vary widely in autonomy and many still output ranked items rather than executing end-to-end tasks.

1.1. A Level-of-Autonomy (LoA) lens for agentic recommender systems

Because “agentic” is used inconsistently across the literature, we adopt a Level-of-Autonomy (LoA) perspective that situates concrete recommender architectures along a continuum from passive ranking to multi-agent orchestration. LoA is driven by four recurring dimensions: (1) **Task scope and planning style**—from single-shot scoring to multi-step plan-act-verify workflows; (2) **Context awareness and memory**—from short in-context state to persistent user/environment memory; (3) **Interaction flexibility**—from static outputs to multi-turn, mixed-initiative, and multi-agent dialogue; (4) **Adaptivity**—from static inference to systems that update profiles, reflect, or learn from feedback. Figure 1 illustrates an L0–L6 spectrum. In this survey, we focus on L2–L5, where concrete agentic architectures exist today and where autonomy-related design choices (memory, tools, orchestration, verification) materially affect recommendation quality and risk.

In this survey, we further classify existing ARS work into **three paradigms**, *i.e.*, agent-assisted recommenders, agent-as-recommenders, and agent-as-simulators. The three paradigms are aligned with the LoA: agent-assisted recommenders often clusters around L2–L4, while agent-as-recommenders and simulators frequently require L4–L5 capabilities such as long-term memory, reflection, and coordination. We detail the dual taxonomy in § 2.

1.2. Task landscape and survey scope

Agentic recommender systems span a broader task space than classical “top- N ranking.” We distinguish (A) tasks directly related to recommendation, (B) tasks that are useful *for* recommendation (training/evaluation/representation), and (C) adjacent non-RecSys agent tasks

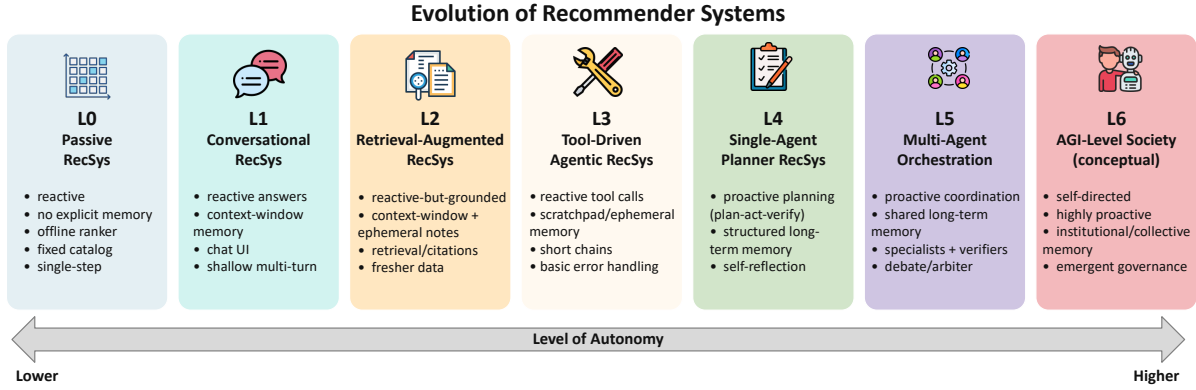


Figure 1 | A Level-of-Autonomy (LoA) spectrum for recommender systems, from passive rankers (L0) to multi-agent orchestration (L5), with L6 as a conceptual “society of agents” endpoint. LoA helps connect architectural ingredients (memory, tools, orchestration) to behavioral capabilities (planning, proactivity, adaptivity) and associated risks.

that sometimes appear in ARS pipelines. Throughout the survey we treat (A) as primary, include (B) when it is used to improve recommendation, and discuss (C) only when it is tightly coupled to recommendation objectives or evaluation.

1.3. Positioning with related surveys

A fast-growing body of surveys has begun to chart LLMs, foundation models, and agents for recommendation, but with different emphases. Broadly, existing works fall into two groups: (i) surveys on LLM-/FM-enhanced recommenders and (ii) surveys that explicitly foreground LLM agents or agentic paradigms. Our survey is positioned at their intersection but adopts a distinct autonomy-centric, RS-specific viewpoint. This survey adopts an explicitly **autonomy-centered, RS-specific** perspective and seeks to unify agentic recommender systems across roles and implementation styles. The comparison with related surveys are summarized in Table 2.

1.4. Search Methodology

The agentic RS field is evolving rapidly; to ensure comprehensive coverage we followed a systematic search strategy. We queried major academic databases and digital libraries, including DBLP, the ACM Digital Library, IEEE Xplore and arXiv, from 2018 through March 2026. Search keywords combined terms from recommender systems and agentic AI, such as “agent recommend”, “RAG recommend”, “retrieval-augmented recommend”, “agent personalization” and “recommend simulator”. Because the technology is new, few papers prior to 2022 discussed LLM agents; nevertheless we included earlier works on conversational RS and multi-agent recommender systems as precursors. Concretely, we collect the papers in the following three steps: 1) **Inclusion criteria**. We included papers involving autonomous or semi-autonomous agents for recommendation, user simulation, or evaluation, while excluding works using foundation models only as feature extractors. 2) **Iterative snowballing**. Starting from a seed set, we expanded the corpus through backward and forward citation tracking until no major new works were found. 3) **Annotation and coding**. Each paper was annotated by agent role, architecture, autonomy level, modality, evaluation, and trustworthiness aspects.

Literature statistics. Based on the annotated results, we visualize the distributions of different agent roles and autonomy levels in Figure 2, from which we have the following observations: (1)

Table 2 | Positioning our survey with respect to existing surveys on LLMs, foundation models, and agentic recommender systems.

Survey	Primary scope	Main perspective	Gap w.r.t. our agentic LoA view
Wu et al. (2024b) (2024)	LLM-based RS	Discriminative vs. generative LLMs; pre-training / fine-tuning / prompting taxonomies	Focuses on LLM models; no explicit autonomy levels or multi-agent orchestration framing.
Zhao et al. (2024b) (2024)	LLM-enhanced RS	End-to-end view across data, model, and application layers	Does not differentiate retrieval-augmented, tool-using, and planning agents along an autonomy axis.
Li et al. (2024) (2024)	Generative recommendation	LLMs as generators that directly output recommended items	Agent memory, tool use, and multi-agent collaboration are largely out of scope.
Huang et al. (2025a) (2025a)	FM-powered RS	Feature-based vs. generative vs. agentic FM integration paradigms	“Agentic RS” is one paradigm; no level-wise mapping to concrete autonomy/design patterns.
Zhang et al. (2025d) (2025)	Agents for RS & search	Role-based taxonomy of LLM agents in IR (interface, optimizer, simulator, etc.)	Jointly covers search and RS; lacks an RS-tailored autonomy framework and level-wise mapping.
Peng et al. (2025) (2025)	LLM agents for RS	Scenario-centric paradigms: recommender-, interaction-, and simulation-oriented agents	Autonomy levels and planning workflows are mostly implicit.
Huang et al. (2025b) (2025b)	Agentic RS perspective	Four-level evolution from static RS to agentic RS; multimodal LLMs and open challenges	Forward-looking; does not systematically catalogue existing ARS across fine-grained autonomy levels and tasks.
Zhu et al. (2025b) (2024/2025)	RS & LLM agents (two-way)	RS for agents and agents for RS; strong focus on trustworthiness	Component- and trustworthiness-centric; lacks a unified LoA mapping of concrete RS architectures and roles.
Maragheh and Deldjoo (2025) (2025)	Multi-agent RS (broad)	Definitions, coordination patterns, and system-level open challenges	Not LLM-specific; complements our RS-specific LoA and role-oriented mapping of existing LLM-driven ARS.

Rapid growth in agentic recommendation. The number of papers increased dramatically from approximately 27–30 in 2024 to 90–93 in 2025 (a roughly 3× growth), indicating that the agentic recommendation field has been attracting significant research interests. (2) **Shift towards goal-driven automatic agentic recommendation.** Agent as Recommender is the dominant paradigm and continues to grow, while Agent as Simulator shows a clear upward trend, rising from 13.3% in 2024 to 24.7% in 2025. (3) **Shift towards higher-autonomy-level recommendation systems.** Level 4 (Single Agent) consistently dominates with over 40%, while Level 5 (Multi-Agent) shows a steady upward trend (from 22.2% in 2024 to 28.6% in 2026).

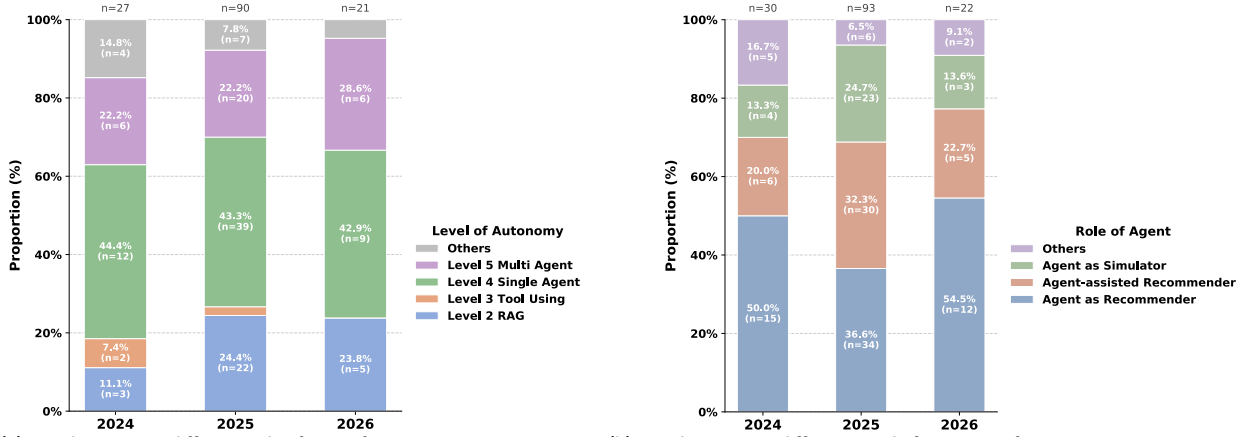


Figure 2 | Statistics of the reviewed literature from 2024 to March 2026. Left: distribution over different levels of autonomy. Right: distribution over different roles of agents.

Core contributions of this survey. Building on the above view, our main contributions are:

1. **An autonomy-centric definition and scope for agentic recommender systems.** We formalize ARS as recommenders that embed LLM-powered agents with memory, tool use, and planning into the recommendation loop, clarifying boundaries with non-agentic LLM-based RS and earlier traditional RS.
2. **A unified taxonomy (*agent role* $\times$ *autonomy level*).** We use this taxonomy to systematically organize papers from the year of 2023 to early 2026 and reveal clusters and gaps across autonomy levels. For each level, we summarize the existing literature by identifying the key components and the high-level capabilities for autonomy across different tasks and scenarios. We also highlight the comparisons between each paradigm (*i.e.*, agent-assisted recommender, agent as recommender, and agent as simulator), and point out the key challenges with open research problems.
3. **A detailed treatment of evaluation and benchmarking for ARS.** We review offline and online metrics, LLM-as-judge protocols, and simulation-based assessments, and motivate agent-specific metrics for planning, tool use, efficiency, and trustworthiness.

Organization. § 2 introduces foundational definitions (agents, tools, memory, workflows) and the taxonomies. Sections 3-4 review agent-assisted recommenders, agent-as-recommenders, and agentic simulators (§ 5) Lastly, § 6 focuses on evaluation and benchmarking, open challenges and future directions.

2. Foundations and Taxonomy of Agentic Recommender Systems

This section unifies two goals: (i) formalize agentic recommendation as an *interactive decision process* and define the entities we study (agents and agentic recommender systems) in §2.1, and (ii) introduce a taxonomy that can organize a rapidly growing literature beyond conventional RS task/method categorizations in §2.3.

2.1. Recommender systems as interactive decision processes

Classical recommender systems are often presented as static mapping functions that produce a ranked list given a user and context. Agentic recommender systems (ARS), in contrast, are more naturally modeled as *interactive decision processes* in which the system repeatedly observes a user’s

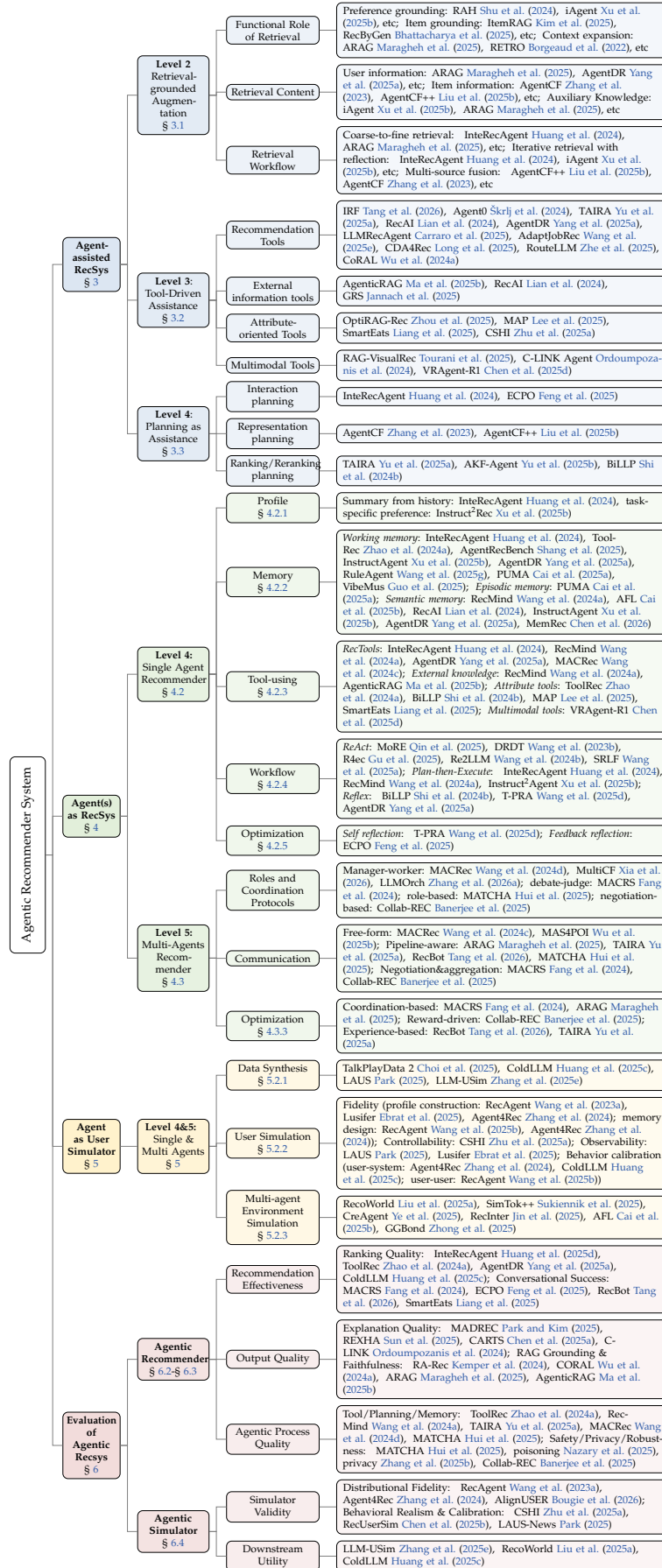


Figure 3 | Overview structure of this survey.

requests and feedback, consults internal state and external resources, and decides what to do next. This shift is not merely a different UI; it changes what should be considered “the recommender”: the recommender becomes an *entity that chooses actions* (e.g., ask a question, retrieve candidates, apply constraints, call tools, verify a plan, generate an explanation), rather than a single-shot ranker.

To make this explicit, we view recommendation as proceeding in discrete turns $t = 1, 2, \dots$. Let u_t denote the user input at turn t (which could be a query, constraints, or feedback). Let $h_t = (u_1, y_1, \dots, u_t)$ denote the interaction history up to the current turn, where y_t is the system output (e.g., recommendation response, clarification question, or tool-triggered result). In agentic settings, the system also maintains an internal state s_t , which includes user/item profiles, memory, and other latent variables required for long-horizon consistency. The system chooses an action a_t from an action space $\mathcal{A}$. Importantly, $\mathcal{A}$ is not restricted to “recommend item i .” It typically includes: *dialogue actions* (ask, confirm, explain, negotiate trade-offs), *tool actions* (retrieve, rank, filter, query a KB, browse the web), *environment actions* (e.g., add to cart, reserve, schedule) when in scope.

In the following, we define the LLM agent and agentic recommender, respectively.

Definition 2.1 (LLM Agent for Recommendation). An LLM agent is an entity that repeatedly (i) **observes** an input comprising user signals and environment/tool outputs, (ii) **reasons** to interpret goals and constraints, (iii) optionally **plans** a sequence of actions, (iv) **acts** by producing user-facing outputs and/or invoking tools, and (v) **updates** its state (profile/memory) based on new evidence.

We represent an LLM agent as $A = (\mathcal{L}_\theta, \pi, \mathcal{P}, \mathcal{M}, \mathcal{T})$, where $\mathcal{L}_\theta$ is the LLM (or foundation model) used as the main reasoning engine. π is the controller/policy that determines how the LLM is prompted, how tool calls are formatted/parsed, and how multi-step execution proceeds (e.g., ReAct). Different workflows are detailed in § 4.2.4. $\mathcal{P}$ is the profile module (persistent traits and summaries of the user and/or items). $\mathcal{M}$ is the memory module (a store of episodic and/or semantic information used across turns and sessions). $\mathcal{T}$ is the tool set (retrieval, ranking services, search engines, KB queries, etc.), along with a tool interface that maps agent outputs to executable calls.

Definition 2.2 (Agentic Recommender System, ARS). An agentic recommender system is an interactive recommendation system that contains one or more LLM agents that can plan and act using tools and memory to produce personalized outcomes. We formalize: $ARS = (\mathcal{U}, \mathcal{I}, \mathcal{E}, \mathcal{R}, \mathcal{A})$, where $\mathcal{U}$ is the set of users, $\mathcal{I}$ is the item universe, $\mathcal{E}$ is the external environment (web, APIs, KBs, simulators), $\mathcal{R}$ denotes classical RS components (optional; present in agent-assisted ARS), and $\mathcal{A} = \{A_1, \dots, A_n\}$ is the set of agents.

This definition is intentionally broad enough to cover three recurring paradigms in the literature: **agent-assisted recommendation**, **agent-as-recommender**, and **agent-as-user-simulator**. In this survey, we describe this role-based partition and ties it to LoA levels, which is detailed in § 2.3.

2.2. Why agentic systems require a new taxonomy

Conventional taxonomies in recommender systems have focused on algorithmic families (matrix factorization versus neural networks), task categories (retrieval, ranking, reranking) or data modalities. While such classifications were adequate for static pipelines, they struggle to describe the emergent landscape of *agentic recommenders*. In these systems the recommender is no longer a single-shot scorer but an entity that can observe, reason, plan and act over multiple turns. Agents can occupy different roles within the recommendation loop—sometimes augmenting a classical model with retrieval or constraint reasoning, sometimes replacing it entirely, and sometimes simulating

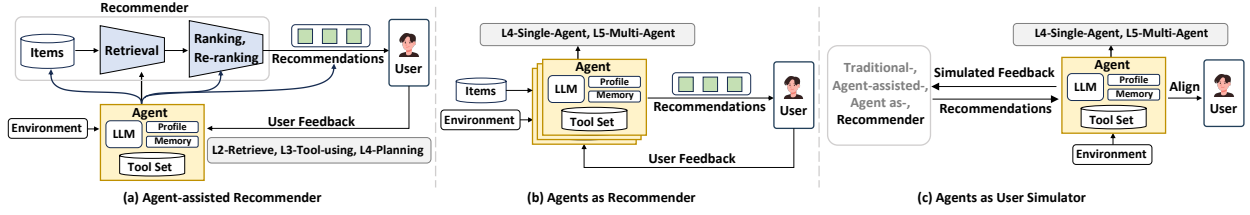


Figure 4 | Illustration of three paradigms of agentic recommender systems. While all of these variations make use of tools, memory, and user profiles, the main difference between (a) and (b) lies in whether they involve a classical retrieval component—*e.g.*, dense retrieval or collaborative-style retrieval as in (a)—or whether this process is handled by a single agent/LLM without the help of auxiliary retrieval modules, as in (b). In (c), agents simulate user behavior and provide simulated feedback to analyze or improve the recommender system. Thus, for example, in this view, we observe an agentic RAG workflow, where the agent is assisted by external tools, an LLM, and memory components to support retrieval and generation within the recommender pipeline.

users or environments to generate feedback for training and evaluation. Two systems trained on the same dataset may therefore differ radically in how they treat an agent: one may rely on an agent merely to translate natural language into search queries, while another may delegate all decision-making to the agent. These differences affect the action space, interaction protocol, and evaluation methodology; they cannot be captured by algorithmic labels alone. A new taxonomy must therefore capture both *where* the agent is positioned within the recommendation pipeline and *what* the agent is allowed to do, providing a framework that unifies agentic augmentation, agentic replacement, and agentic simulations.

2.3. A two-axis taxonomy: roles and autonomy levels

To organise the burgeoning literature, we propose a two-dimensional taxonomy that cross-classifies agentic recommenders by **macro role**—the agent’s placement in the recommendation loop—and by **autonomy level**—the breadth of actions the agent can perform and the capabilities the agent can achieve. This structure clarifies relationships among systems that otherwise share similar datasets or metrics.

Macro roles. We distinguish three high-level roles as illustrated in Figure 4:

1. **Agentic augmentation.** The agent works alongside a classical recommender, improving specific stages such as preference elicitation, evidence retrieval, or result filtering. The classical model still produces the final ranking. Systems like *RecMind* and *iAgent* fall here, where the agent interprets natural-language queries and invokes retrieval or ranking tools but does not itself decide the top items.
2. **Agentic replacement.** One or more agents assume full responsibility for recommendation. They maintain user state, orchestrate tool calls, retrieve and rank items, and generate explanations. Classical models, if present, appear as tools rather than the primary controller. Examples include *InteRecAgent*, *MADREC* and *DRDT*, where an LLM agent decides both what to recommend and how to justify it.
3. **Agentic simulation.** Agents model the behaviour of users or environments, generating clicks, ratings, critiques or dialogue acts for training, evaluation and robustness analysis. Multi-agent environments such as *RecoWorld* and *LAUS-News* allow experiments with complex user-item dynamics without deploying on real users.

Autonomy levels. Orthogonal to role, autonomy measures how broadly an agent can act. This survey focuses on:

1. **L2 — Retrieval-grounded assistance.** The agent supplements a classical system by retrieving relevant evidence—user history, item attributes, domain knowledge—but does not orchestrate tools or plan. This tier includes retrieval-augmented generation frameworks such as ARAG that ground LLM outputs in evidence to reduce hallucinations.
2. **L3 — Tool-orchestrating assistance.** The agent selects and invokes specialized tools implementing subroutines (retrievers, rankers, constraint solvers, search engines, multimodal handlers) and may choose among them. ToolRec and iAgent exemplify this level.
3. **L4 — Single-agent planning.** A single agent performs multi-step reasoning: it decomposes tasks, calls tools sequentially, reflects on intermediate results and self-corrects errors. Controllers may follow REACT loops, planner-executor frameworks or reflection mechanisms. *InteRecAgent* and *Thought-Augmented Planning* are characteristic examples.
4. **L5 — Multi-agent orchestration.** Decision authority is distributed across specialised agents—planners, retrievers, rankers, critics, explainers and safety monitors—that communicate via explicit protocols. Manager-worker, debate-judge and negotiation schemes allow agents to coordinate. Systems such as *MACRec* Wang et al. (2024c), *MACRS* Fang et al. (2024), *Collab-REC* Banerjee et al. (2025), *RouteLLM* Zhe et al. (2025) and *MATCHA* Hui et al. (2025) illustrate this tier.

Crossing these axes reveals the diversity of agentic systems. For instance, a retrieval-augmented assistant that re-ranks items from a classical model belongs to the “agentic augmentation” macro role with L3 autonomy, whereas a multi-agent simulation environment that models users and producers belongs to the “agentic simulation” role with L5 autonomy. This cross-classification clarifies both how agents interact with classical components and how much autonomy they exercise.

3. Agentic Augmentation: Agent-Assisted Recommender Systems

Agent-assisted recommender systems insert an LLM agent *around* a pre-existing recommendation pipeline to *augment* specific sub-tasks (e.g., retrieval/grounding, intent understanding, explanation, or constraint enforcement), while the core recommender remains the primary decision module. In the language of autonomy, this category largely subsumes **L2 (retrieval-augmented)** and **L3 (tool-using)** systems: the agent improves the pipeline without fully replacing it, and the output is typically still anchored to classic recommendation primitives such as candidate generation and ranking. This “assistance” framing matters because it changes what we optimize and what we measure: rather than asking whether an agent can recommend end-to-end, we ask which *interventions* improve user and system outcomes, and how to evaluate those interventions reliably.

What tasks do agents assist with? Across the recent literature, agent-assisted systems repeatedly target a small set of recurring tasks, which can be grouped by *where* they intervene in the recommendation loop:

- **Understanding and control:** interpreting natural-language intent and constraints, eliciting missing preferences, and translating user commands into structured control signals (e.g., conversational steering, controllable constraints) Chen et al. (2025c); Feng et al. (2025); Huang et al. (2025d); Luo et al. (2025); Tang et al. (2026); Zeng et al. (2024b).

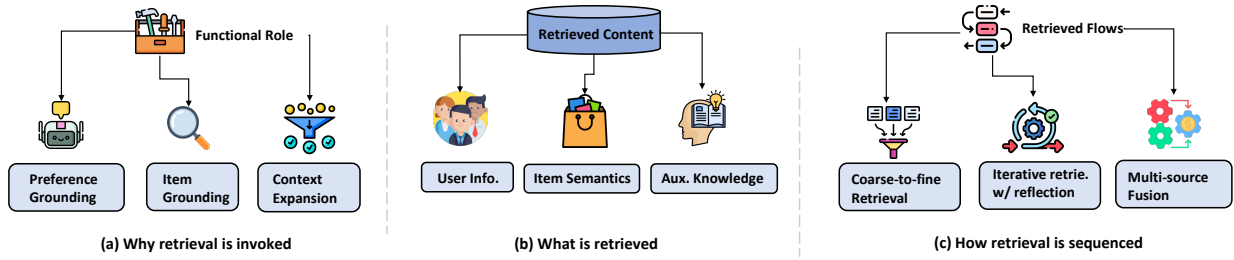


Figure 5 | Illustration of three aspects of Retrieval-Augmented Assistance for Agent-assisted Recommender Systems.

- **Grounding and knowledge access:** retrieving relevant information to reduce hallucinations and increase specificity (user-history snippets, item semantics such as attributes/reviews, external knowledge such as domain facts), typically via RAG-style components [Feng et al. \(2025\)](#); [Huang et al. \(2025d\)](#); [Tang et al. \(2026\)](#).
- **Candidate operations and reranking:** generating candidate sets, filtering candidates under constraints, and reranking by richer semantic criteria (*e.g.*, compatibility reasoning or cross-domain transfer), often by coupling agents to RecTools [Alamri et al. \(2025\)](#); [Liu et al. \(2025b\)](#); [Yang et al. \(2025a\)](#); [Zhao et al. \(2024a\)](#).
- **Explanation and interaction quality:** producing grounded explanations and multi-turn justifications; summarizing rationales; clarifying trade-offs; adapting explanations to context [Park and Kim \(2025\)](#); [Sun et al. \(2025\)](#).
- **Governance:** enforcing safety/fairness constraints, inserting intermediate representations or shields, and providing “guardrails” that sit between user, agent, and recommender [Xu et al. \(2025b\)](#); leveraging agent as engineer to update the recommendation strategies ?.

Tasks often span multiple stages. For example, a multi-turn conversational assistant may (i) interpret an instruction, (ii) retrieve relevant history, (iii) call a ranking tool, and (iv) generate a justification; tool use and retrieval become *mechanisms* that enable the task rather than ends in themselves.

3.1. Retrieval-Augmented Assistance

Retrieval augmentation is the dominant pattern for L2 systems and a key building block for L3 systems. While RAG is broadly used in NLP, in recommender systems the retrieved evidence must align with recommendation structure: it may contain *user-side* information (profiles, histories, constraints), *item-side* information (descriptions, reviews, attributes), or *auxiliary knowledge* (domain facts, knowledge graphs, taxonomies). Following the original survey’s framing, it is useful to organize retrieval-augmented recommendation through three complementary lenses:

(i) **Functional role:** why retrieval is invoked. Common functional roles include: (1) **Preference grounding** (retrieve history snippets to ground intent and reduce inconsistency across turns). For example, RAH [Shu et al. \(2024\)](#) enhances the retrieval stage by querying a structured personality library to extract user-specific preferences, goals, and value orientations. These retrieved signals are then used to filter and prioritize candidate items, enabling more accurate and personalized candidate generation before downstream ranking. iAgent [Xu et al. \(2025b\)](#) retrieves external knowledge through tool invocation while simultaneously leveraging the internal knowledge of LLMs, thereby assisting the reranker in further refining the ordering of an already ranked item list. (2) **Item grounding** (retrieve item evidence to support explanations and avoid fabrication) [Bhattacharya et al. \(2025\)](#); [Kim et al. \(2025\)](#), and (3) **Context expansion** (retrieve domain knowledge for niche domains such as health, finance, or travel) [Banerjee et al. \(2024\)](#); [Borgeaud et al. \(2022\)](#); [Cho](#)

et al. (2025); Maragheh et al. (2025); Rao and Lin (2024); Zhao et al. (2025). This perspective clarifies what “success” means: *e.g.*, for preference grounding we care about stability and faithfulness to user history, whereas for context expansion we care about factuality and usefulness.

(ii) **Retrieved content**: what is retrieved. Retrieved content typically falls into: **user information** (profiles, histories, constraints) Ao et al. (2025); Huang et al. (2024); Kim et al. (2026); Maragheh et al. (2025); Ning et al. (2026); Shu et al. (2024); Wang et al. (2024a); Wei et al. (2025a); Yang et al. (2025c); Yu et al. (2025b). User information is typically divided into long-term and short-term preferences and represented as interaction histories or textual preference evidence. For example, ARAG Maragheh et al. (2025) collects textual cues from both the current session and long-term logs to summarize user intent and support re-ranking. AgentCF Zhang et al. (2023) retrieves user memory and item memory to enrich preference representations and item semantics. AgentDR Yang et al. (2025a) further incorporates tool-applicability memory, improving the automation level of agent-assisted recommendation. **item semantics** (attributes/reviews; multimodal content where available) Balachandran and Masum (2025); Kieu et al. (2025); Kim et al. (2025); Li et al. (2025); Liu et al. (2025b); Qiu et al. (2025); Wang et al. (2025f); Wei et al. (2025b); Xu et al. (2025a); Yang et al. (2025d); Zhang et al. (2023); for example, AgentCF Zhang et al. (2023) retrieves item memory to enrich item-side semantics, while AgentCF++ Liu et al. (2025b) further leverages cross-domain semantic evidence to improve candidate understanding. and **auxiliary knowledge** (external corpora, KGs, or curated domain sources) Borgeaud et al. (2022); Maragheh et al. (2025); Meng et al. (2025a); Xu et al. (2025b); for example, iAgent Xu et al. (2025b) queries outside knowledge sources to refine reranking. This choice affects privacy and leakage risk: user-side retrieval often contains sensitive personal data, while auxiliary knowledge retrieval raises provenance and licensing issues.

(iii) **Retrieval flows**: how retrieval is sequenced. Beyond “retrieve once then generate,” recent systems increasingly use multi-step flows, such as: 1) coarse-to-fine retrieval (retrieve broad candidates, then refine by constraints) Huang et al. (2024); Li and Lim (2025); Maragheh et al. (2025), 2) iterative retrieval with reflection (retrieve, draft, detect missing evidence, retrieve again) Huang et al. (2024); Xu et al. (2025b); Zhang et al. (2025c), or 3) multi-source fusion (combine user-history retrieval with item-attribute retrieval) Liu et al. (2025b); Zeng et al. (2024a); Zhang et al. (2023). These flows implicitly introduce *planning* and should be evaluated at the trajectory level, not only by final accuracy.

Key limitations. RAG-based assistance can still fail via (i) *retrieval bias* (frequent or easy-to-retrieve patterns dominate), (ii) *context overload* (critical evidence is “lost in the middle”), and (iii) *spurious grounding* (retrieved text gives false confidence without causal relevance) Cheng et al. (2025); Lichtenberg et al. (2024); Lin et al. (2025); Liu et al. (2024); Maragheh et al. (2025); Meng et al. (2025b); Sayana et al. (2025); Sun et al. (2025); Sun (2025); Tandon and Banerjee (2025); Xu et al. (2025b); Zhang et al. (2025a). These motivate evaluation protocols that explicitly test robustness to retrieval noise and long-context failure (Sec. 6). Compared with conventional retrieval systems, retrieval-augmented recommender systems incorporate external behavioral traces and world knowledge into the recommendation process, thereby enhancing personalization and generalization. However, their autonomy remains limited: while they are able to integrate long-term context and historical memory to some extent, they still primarily operate reactively rather than proactively, rely on single-turn retrieval with limited interaction flexibility, and utilize fixed pipelines that are difficult to adapt. Thus, retrieval-augmented recommender systems can be regarded as a transitional stage bridging static retrieval models and autonomous recommendation agents capable of strategic reasoning, context-aware planning, and adaptive learning.

Table 3 | Unified Summary of Retrieval-Augmented Assistance, Tool-Driven Assistance, and Planning as Assistance.

Category	Dimension	Sub-type	Representative Works	High-level Capabilities
Retrieval-Augmented Assistance	Functional Role (Why retrieval is invoked)	Preference Grounding	RAH Shu et al. (2024), iAgent Xu et al. (2025b)	Proactivity: Reactive Context Awareness: Memory (long-/short-term) Interaction: Single-turn retrieval Adaptivity: Static pipeline
		Item Grounding	AgentCF Zhang et al. (2023), AgentCF++ Liu et al. (2025b)	
		Context Expansion	ARAG Maragheh et al. (2025), iAgent Xu et al. (2025b)	
	Retrieved Content (What is retrieved)	User Information	InteRecAgent Huang et al. (2024), RAH Shu et al. (2024), RecMind Wang et al. (2024a), ARAG Maragheh et al. (2025), AgentCF Zhang et al. (2023), AgentDR Yang et al. (2025a)	
		Item Semantics	AgentCF Zhang et al. (2023), AgentCF++ Liu et al. (2025b)	
		Auxiliary Knowledge	ARAG Maragheh et al. (2025), iAgent Xu et al. (2025b)	
Tool-Driven Assistance	Tool Taxonomy (What tools are used)	Coarse-to-Fine	InteRecAgent Huang et al. (2024), ARAG Maragheh et al. (2025)	Proactivity: Proactive (tool invocation) Context Awareness: Internal & external memory Interaction: Multi-turn Adaptivity: Adaptive strategy
			InteRecAgent Huang et al. (2024), iAgent Xu et al. (2025b)	
		Iterative with Reflection	AgentCF Zhang et al. (2023), AgentCF++ Liu et al. (2025b)	
		Multi-source Fusion	InteRecAgent Huang et al. (2024), ToolRec Zhao et al. (2024a)	
		RecTools	iAgent Xu et al. (2025b), RecMind Wang et al. (2024a)	
		External Info Tools	ToolRec Zhao et al. (2024a)	
Planning as Assistance	Interaction Planning	Attribute-oriented Tools	HuggingGPT Shen et al. (2023), Toolformer Schick et al. (2023)	Planning: Goal decomposition, tool orchestration Reasoning: Multi-step reasoning, reflection Adaptivity: Dynamic memory & plan updates Context Awareness: Tools, history, interactions
		Multimodal Tools		
	Representation Planning	Reflective Plan Revision	InteRecAgent Huang et al. (2024)	
		Expectation-driven Steering	ECPO Feng et al. (2025)	
	Ranking/Reranking Planning	Semantic Enrichment	AgentCF Zhang et al. (2023)	
		Cross-domain Transfer	AgentCF++ Liu et al. (2025b)	
Planning as Assistance	Ranking/Reranking Planning	Reasoning-driven Reranking	Intelligent Agent Yu et al. (2025b)	Planning: Goal decomposition, tool orchestration Reasoning: Multi-step reasoning, reflection Adaptivity: Dynamic memory & plan updates Context Awareness: Tools, history, interactions
		Reflective Learning	Re2LLM Wang et al. (2024b), MoRE Qin et al. (2025), SRLF Wang et al. (2025a), DRDT Wang et al. (2023b)	
	Ranking/Reranking Planning			

3.2. Tool-Driven Assistance

Tool use is the hallmark of L3 systems: the agent does not merely condition on retrieved text, but *acts* via API calls, database queries, retrieval services, rerankers, and other components. In recommendation, tool use spans different stages, such as retrieval, ranking, and reranking.

Tool taxonomies and design choices. A practical taxonomy separates: (i) **RecTools** directly provides candidate sets or ranking lists (e.g., traditional retrievers, rankers, candidate filters models); (ii) **external information tools** retrieve external knowledge and contextual signals (e.g., web/domain search, knowledge base query tools); (iii) **attribute-oriented tools** filter items based on structured attributes or specific constraints (e.g., feature calculators, KG completion, constraint checkers); and (iv) **multimodal tools** are used to understand multimodal features, or generate multimodal content to supplement recommendation tasks (e.g., vision/audio encoders, captioners) Huang et al. (2024); Schick et al. (2023); Shen et al. (2023); Shi et al. (2025); Yang et al. (2025a); Zhao et al. (2024a). These tools can be invoked separately or sequentially in different contexts. For example, InteRecAgent Huang et al. (2024) calls recommendation tools to support task planning and execution, progressively refining the candidate pool before returning it to the LLM, which then delivers the final response to the user. In re-ranking, iAgent Xu et al. (2025b) queries external knowledge through search tools and forwards the retrieved information to a re-ranker to refine an already-ranked list. AgentDR exemplifies a tool-centric view where the agent learns to uncover implicit item-item relations by invoking a pipeline of tool operations, enabling dynamic recommendation under changing contexts Yang et al. (2025a). Tool selection policies can be hand-coded, prompted, or learned; recent work on modular “hooks” attempts to decouple tool invocation logic from prompts, improving maintainability and safety auditing de Mijolla et al. (2024). In practice, tool-driven assistance often succeeds not because the LLM is a strong ranker, but because it orchestrates structured components with better inductive bias (retrieval, ranking, graph search).

3.3. Planning as Assistance

Planning arises naturally once agents operate over multi-turn conversations and multi-step tool sequences. Importantly, planning is not only about conversation; it can appear in: (1) **interaction planning** (e.g., what to respond next, how to improve the recommendations) can be injected to decide what to ask next to the user, or be leveraged to generate intermediate plans to give guidance signals to improve recommender models that can align better with the user needs Feng et al. (2025); Huang et al. (2024). (2) **Rrepresentation planning** (e.g., what and how to update user preference and item descriptions) allows LLMs to construct richer, updated, and explainable user and item descriptions Liu et al. (2025b); Xie et al. (2026); Zhang et al. (2023). (3) **Ranking/re-ranking planning** usually considers what constraints to satisfy first, or how to trade-off objectives, thus leading to a more constraint-compatible result that aligns with user preference at the same time Shi et al. (2024a); Yu et al. (2025a,b). (4) **Optimization planning** leverages LLM agents to automatically diagnose algorithm limitations and update the recommendation strategies/pipelines iteratively Lao et al. (2026); Wang et al. (2026a).

Existing literature has explored different types of planning to augment recommender systems. For example, InteRecAgent Huang et al. (2024) introduces explicit planning into the recommendation pipeline. By reflecting on user interactions, tool invocation plans, and intermediate recommendation outcomes, the agent continuously revises and optimizes its internal plans, thereby enhancing the system’s understanding of user intent and enabling more controllable and self-improving interactions. To enhance user/item representations, AgentCF Zhang et al. (2023) leverages automated user interaction histories and textual feedback to enrich the semantic representations of both users and items. By incorporating global collaborative information, it produces more comprehensive and fine-grained embeddings that offer a stronger foundation for downstream recommendation. In the ranking task, the Intelligent Agent Yu et al. (2025b) applies predefined reasoning rules to the pre-ranked list, performing step-by-step inference to assess whether each candidate item truly aligns with the user’s explicit or implicit preferences. This

reasoning-driven refinement yields a more accurate and user-aligned final ranking than traditional scoring-based approaches.

Comparison between tool-driven and planning assistance (L3 $\rightarrow$ L4). Compared with tool-driven agents, the single-agent planner stage exhibits stronger autonomy: it can actively set goals, break them into steps, and continuously observe, reflect, and adjust during execution; Together, these capabilities enable richer context awareness, proactive decision-making, flexible interaction, and continuous adaptivity, resulting in improved generalization and long-term recommendation performance.

4. Agentic Replacement: Agent(s) as Recommender

4.1. Definition and Scope

The paradigm *agent as recommender* refers to systems in which an intelligent agent directly assumes the role of the recommender rather than merely supporting a conventional recommendation pipeline. In such systems, the agent is the end-to-end decision maker that (i) interprets user intent, (ii) maintains user state over time, (iii) selects and executes actions that may include tool invocation and external information gathering, and (iv) produces the final recommendation and natural-language response. The distinguishing feature is *decision authority*: classical recommendation models, if present, are invoked as tools rather than acting as the primary controller [Huang et al. \(2024\)](#); [Wang et al. \(2024a\)](#).

To systematize the diverse methodologies in agent as recommender, we categorize existing works into two primary architectural paradigms: **Single-Agent** and **Multi-Agent**. While both paradigms share the fundamental components of agents (such as Profile, Memory, Tool-Using, Workflow, and Optimization), they diverge fundamentally in how these components are orchestrated and how decision-making authority is distributed.

Single-Agent based Recommender functions as a centralized, monolithic decision-maker. In this paradigm, a single agent acts as the unified “brain,” responsible for the entire lifecycle of the recommendation process—from interpreting user intent and managing internal memory to invoking tools and generating the final response. The core challenge here lies in enhancing the individual agent’s capabilities to handle complex reasoning chains without losing context or coherence. As detailed in § 4.2, our discussion focuses on how individual modules (e.g., Profile, Memory) are internalized within a single entity to achieve proactivity and adaptivity.

Multi-Agent based Recommender, in contrast, operates as a decentralized collaborative ecosystem. Here, the recommendation task is decomposed into sub-problems assigned to specialized agents (e.g., a Planner, a Critic, or a Domain Expert). The system’s intelligence emerges not from a single model’s depth but from the interaction between agents. This paradigm addresses the limitations of single-agent contexts by distributing cognitive load and introducing mechanisms like debate and role-playing. Consequently, in § 4.3, our focus shifts from internal modules to inter-agent dynamics, specifically emphasizing *Communication* protocols and collective *Optimization*.

This taxonomy allows us to dissect how the five core components evolve: roughly speaking, what manifests as an internal *Workflow* in a single agent often scales into a *Communication* protocol in a multi-agent system; similarly, internal *Self-Reflection* evolves into peer-to-peer *Feedback*. In the following, we first elaborate on *single-agent based recommender*, discussing how the five core components are internalized within a single agent to achieve proactivity and adaptivity for recommendation (§4.2). Then, we shift from internal modules to inter-agent dynamics and focus

on *Multi-Agent based Recommender* (§4.3), specifically emphasizing *Communication* protocols and collective *Optimization*.

4.2. Single-Agent Recommenders (LoA L4)

Modeling recommender as a single agent substantially upgrades their high-level capabilities beyond what traditional model-centric pipelines can offer. 1) As a unified decision-making entity, a single agent enables *proactivity*, transitioning the system from purely reactive responses to actively inferring needs, initiating clarifications, and planning ahead within a session. 2) It also enhances *context awareness* by maintaining a coherent internal state: while the underlying model is constrained by a finite context window, the agent can leverage explicit memory or state tracking to preserve user preferences and interaction history across turns. 3) Moreover, the single-agent paradigm improves *interaction flexibility*, supporting multi-turn, natural-language preference elicitation and iterative refinement rather than single-shot prediction. 4) Finally, a single agent introduces *stronger adaptivity*: it can continuously update its internal beliefs, adjust its recommendation strategy based on ongoing feedback, and self-correct within a session, instead of acting as a static mapping from input to output. Together, these improvements make the single-agent recommender more conversational, contextually grounded, and behaviorally adaptive than traditional recommender systems.

To systematize the literature, we follow a reusable decomposition into five components: profile (§4.2.1), memory (§4.2.2), tool-using (§4.2.3), workflow (§4.2.4), and optimization (§4.2.5). This decomposition is practical because it enables comparison between systems that differ in task setting (e.g., conversational versus sequential recommendation) but share common control-loop structures.

4.2.1. Profile: Persistent Preference and Constraint Modeling

The *profile* is a persistent representation capturing long-term preferences, constraints, and relatively stable aspects of user intent. Unlike classical recommenders where profiles are often implicit (encoded in latent embeddings), agentic recommenders frequently maintain explicit textual or structured profiles to support interpretability and controllability Huang et al. (2024); Tang et al. (2026). Profiles can be constructed by summarizing interaction histories into preference statements, maintaining structured constraint lists (budget, time, category exclusions), or extracting higher-level values that explain the user’s choices.

In agentic recommender systems, the profile serves as the foundation for modeling both users as autonomous entities with distinct goals, preferences, and behavioral patterns. For instance, in *InteRecAgent* Huang et al. (2024), the profile module maintains a structured user representation comprising three facets (i.e., like, dislike, and expect) which are dynamically synthesized by the LLM from conversational history to capture both long-term preferences and short-term intentions for tool invocation. *Instruct²Agent* Xu et al. (2025b) maintains an instruction-aware profile updated with round-wise user feedback and a dynamic extractor that derives task-specific preferences under current instructions, enabling per-user optimization decoupled from other users.

4.2.2. Memory: Working, Episodic, and Semantic State

Memory is a core component that supports contextual reasoning and provides grounding evidence that constrains the agent’s outputs. A useful abstraction distinguishes: (i) *working memory* (short-term conversational context), (ii) *episodic memory* (retrievable records of prior interactions and feedback), and (iii) *semantic memory* (abstracted preference facts and world knowledge) Liu et al. (2025b); Wang et al. (2024a). Memory can also take different forms, including **textual** (explicit) representations, **parametric** (implicit) representations, or none.

Table 4 | Taxonomy of memory mechanisms and representative works in agentic recommender systems.

Memory Type	Representative Work	High-level Capabilities
Working Memory	InteRecAgent Huang et al. (2024) , ToolRec Zhao et al. (2024a) , AgentRecBench Shang et al. (2025) , InstructAgent Xu et al. (2025b) , AgentDR Yang et al. (2025a) , RuleAgent Wang et al. (2025g) , PUMA Cai et al. (2025a) , VibeMus Guo et al. (2025) , Sunnie Wu et al. (2024c)	Context Awareness: Memory Interaction Flexibility: Strengthen multi-turn interaction Adaptivity: Continuous evolving
Episodic Memory	PUMA Cai et al. (2025a)	
Semantic Memory	RecMind Wang et al. (2024a) , AFL Cai et al. (2025b) , RecAI Lian et al. (2024) , InstructAgent Xu et al. (2025b) , AgentDR Yang et al. (2025a) , MemRec Chen et al. (2026)	

We summarize the three types of memory and how it achieves different agentic capabilities in Table 4, while we give some concrete examples as follows. In InteRecAgent [Huang et al. \(2024\)](#), the memory module is realized as a Candidate Bus, which maintains current item candidates separately from the prompt by combining a data bus—initialized at each conversation turn with all items or user-specified candidates and updated after each tool execution—and a tracker that logs each tool’s input, output, and execution status, enabling sequential streaming of candidates through multiple tools and supporting the reflection mechanism for judgment. RecMind [Wang et al. \(2024a\)](#) defines a Memory component split into: Personalized Memory, which stores individual user data (e.g., their ratings or reviews), and World Knowledge, which stores item metadata (domain-specific) and real-time information via web search. BiLLP [Shi et al. \(2024b\)](#) designed dedicated working memory modules for each of its key components (e.g., actor, critic, and planner) to store past experiences, with memory updated after each step. The memory of [Yu et al. \(2025b\)](#) selectively stores relevant item features, user reviews, and interaction history to support accurate and personalized recommendations continuously.

4.2.3. Tool-Using: Expanding the Action Space.

A key characteristic of agentic systems is their ability to augment reasoning through external tools, enabling agents to dynamically interact with APIs, databases, search engines, or specialized modules to enhance their recommendation capabilities. We categorize the tools employed by existing agents into four main types: (1) **RecTools**: Tools for retrieval, ranking, or classical recommender system engines, which form the core mechanism for filtering relevant candidates and predicting user preferences; (2) **External knowledge**: Tools that access knowledge bases or search engines beyond the recommendation system, supporting responses to user queries that require real-time or external information [Guo et al. \(2024\)](#); (3) **Attribute tools**: Schema-aware or faceted filters for attribute-oriented retrieval, allowing fine-grained control over item selection; (4) **Multimodal tools**: Tools designed to handle additional modalities such as images, audio, or code, extending the agent’s capability beyond textual information.

For instance, in InteRecAgent [Huang et al. \(2024\)](#), retrieval and ranking tools serve as the core recommendation engines by identifying relevant candidates and estimating user preferences, while information query tools allow the agent to access external knowledge bases or databases via SQL and search engines to answer user inquiries beyond standard recommendation tasks. RecMind [Wang et al. \(2024a\)](#) mainly integrates three types of tools to access and process external knowledge: a

Table 5 | Taxonomy of workflow controllers in agentic recommendation systems.

Workflow Type	Representative Work	High-level capabilities
ReAct	RecMind Wang et al. (2024a), ToolRec Zhao et al. (2024a), AgentRecBench Shang et al. (2025), PUMA Cai et al. (2025a), WeMusic-AgentBi et al. (2025)	Proactivity: proactive & reactive,
Plan-then-Execute	InteRecAgent Huang et al. (2024), AFL Cai et al. (2025b), RecAI Lian et al. (2024), AgentRecBench Shang et al. (2025), InstructAgent Xu et al. (2025b), CogRec Hu et al. (2025), ScienceDB AI Long et al. (2026), AMEM4Rec Nguyen et al. (2026)	Interaction Flexibility: Multi-turn, Adaptivity: continuous evolving
Reflex	AgentRecBench Shang et al. (2025), AgentDR Yang et al. (2025a), RuleAgent Wang et al. (2025g)	

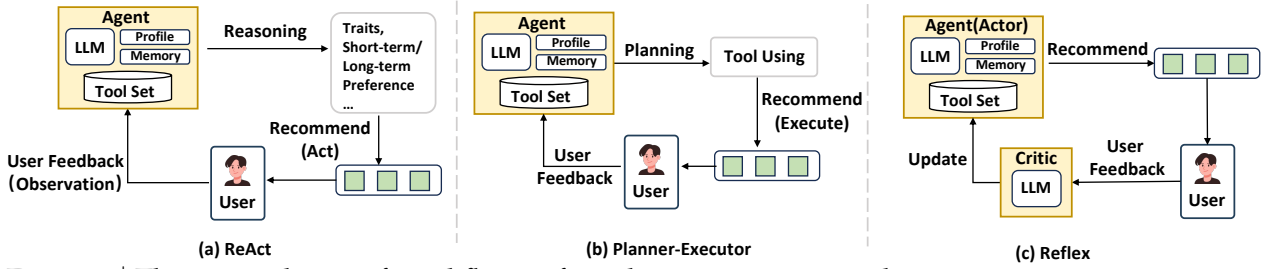


Figure 6 | Three paradigms of workflows of single-agent recommenders.

Database Tool that converts natural language queries into SQL to retrieve in-domain knowledge such as user reviews or item metadata; a Web Search Tool for fetching real-time information from the internet as part of world knowledge; and a Text Summarization Tool to process and condense the retrieved information. In the multi-agent collaboration framework proposed by MACRec Wang et al. (2024c), the Analyst is granted access to two tools to assist in analysis: an information database to get the user profile and item attributes and an interaction retriever to get the user/item interaction history. ToolRec Zhao et al. (2024a) introduces attribute-oriented retrieval and ranking tools: retrieval tools first return candidate items based on specified attribute patterns and size constraints, while ranking tools use LLMs with instruction templates to order these candidates according to user history and attribute relevance, effectively capturing users' latent intent without training separate models for each attribute. Shi et al. (2024b) define a Category Analysis Tool which can identify a list of categories associated with each legal action and conduct a statistical analysis on the user's viewing history. AgentDR Yang et al. (2025a) bridges LLM reasoning with scalable rec tools by delegating full-catalog ranking to traditional recommenders while using the agent to integrate multiple model outputs (personalized tool suitability) and inject commonsense relational reasoning over substitutes/complements grounded in the user's history—mitigating hallucination and keeping the system scalable.

4.2.4. Workflow Controllers: From Reasoning-Acting to Reflection

The workflow controller specifies how the agent sequences reasoning steps and actions. As illustrated in Figure 6, three recurring patterns are prominent. Table 5 summarizes these workflow paradigms, along with representative methods and their associated high-level capabilities. Overall, ReAct emphasizes flexible interaction and incremental reasoning, Plan-then-Execute improves controllability and structured decision-making, while Reflex enables continuous policy refinement

through self-evaluation. These complementary designs collectively support key agentic capabilities, including proactivity, multi-turn interaction flexibility, and adaptive evolution.

- ReAct (interleaved reasoning and acting).** In this pattern, the agent alternates between intermediate reasoning and tool invocation, supporting incremental refinement of candidates and constraints [Yao et al. \(2022\)](#). This is effective for open-ended requests but can compound errors if early tool calls are misguided. A large portion of existing work lies in this workflow. MoRE [Qin et al. \(2025\)](#) introduces a reflection-based framework for sequential recommendation, where an LLM dynamically selects and combines multiple reflectors to iteratively reconsider and correct its recommendation decisions, mitigating biases and instability caused by a single reasoning path. DRDT [Wang et al. \(2023b\)](#) uses dynamic reflection with divergent thinking within a retriever–reranker framework to iteratively probe, critique, and tailor LLM reasoning to better model sequential user preferences over time. R4ec [Gu et al. \(2025\)](#) establishes a reasoning–reflection–refinement loop where an actor model proposes recommendations and a reflection model evaluates and feeds back corrections, enabling deliberate System-2-like thinking to improve recommendation accuracy. Re2LLM [Wang et al. \(2024b\)](#) guides LLMs to self-reflect on recommendation errors to build a hint knowledge base and trains a lightweight agent to select useful hints that correct reasoning, resulting in more accurate session-based recommendations without fine-tuning the LLM. SRLF [Wang et al. \(2025a\)](#) introduces a set-wise reflective learning framework that uses an LLM agent to assess entire candidate item sets, detect mismatches with user feedback, and iteratively refine both user preferences and item semantics through a closed-loop reflection process, capturing richer inter-item relationships for better sequential recommendation.
- Plan-then-Execute.** Here, a planner decomposes the user request into subgoals (*e.g.*, constraint inference, candidate retrieval, verification) and an executor carries out tool calls under this plan. This improves controllability because plans can be inspected and constrained [Huang et al. \(2024\)](#); [Yu et al. \(2025a\)](#). For example, InteRecAgent [Huang et al. \(2024\)](#) adopts a plan-first execution workflow consisting of two stages: in the first stage, the LLM generates a complete tool-utilizing plan based on the user query; in the second stage, the system executes each operation sequentially according to the plan, without invoking the LLM at every step. RecMind [Wang et al. \(2024a\)](#) introduces a Self-Inspiring (SI) planning method that leverages all previously explored reasoning branches to generate each step, enabling more comprehensive and multi-perspective reasoning for recommendation tasks than traditional single-path approaches like CoT [Wei et al. \(2022\)](#) and ToT [Yao et al. \(2023\)](#). Instruct²Agent [Xu et al. \(2025b\)](#) follows plan-then-execute (parser → knowledge/tool use → reranker) with a reflex-style self-reflection that regenerates constrained lists when inconsistencies are detected.
- Reflex:** Self-critique loop or judge–checker pattern [Wu et al. \(2025a,c\)](#). BiLLP [Shi et al. \(2024b\)](#) follow the actor-critic framework and initialize the actor and critic as using LLM, to provide personalized recommendations, where Critic assesses the user’s current satisfaction level (action advantage value) and updates the policy of Actor to enhance personalized recommendations. T-PRA [Wang et al. \(2025d\)](#) adopts an actor–critic framework to balance the trade-off between user satisfaction and user interest exploration in proactive conversational recommendation. AgentDR [Yang et al. \(2025a\)](#) follows plan-then-execute with a reflex-style check: the agent plans which base recommenders to consult, aggregates their candidates, reasons about item-item substitutes/complements, and re-scores candidates before output—acting as a controller atop conventional rankers rather than a monolithic generator.

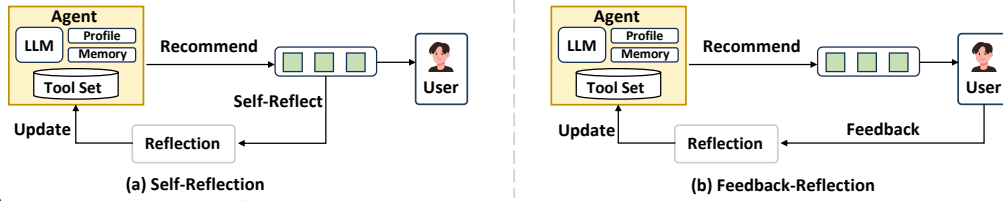


Figure 7 | Two paradigms of optimization in single-agent recommenders.

4.2.5. Optimization and Adaptation

To continuously enhance the decision-making capabilities, recommendation agents must perform optimization over time—refining their internal policies and strategies through iterative learning. Such optimization processes can be broadly categorized into two complementary paradigms: (1) **Self reflection**, where the agent evaluates and improves upon its own reasoning and recommendations; and (2) **Feedback reflection**, where the agent adapts based on external signals from the environment or user feedback. Together, these mechanisms enable agents to move beyond static policy learning toward agentic optimization, characterized by proactive self-improvement and adaptive alignment with long-term user goals.

Self-Reflection: Self-reflection focuses on internal critique and self-improvement loops, where the agent evaluates the quality of its own decisions and refines its policy accordingly. For instance, T-PRA Wang et al. (2025d) introduces a critic module that generates structured feedback on recommendation outcomes. This feedback is then leveraged via Direct Preference Optimization (DPO) to jointly refine both the actor and advisor components, improving alignment with long-term proactive recommendation objectives.

Feedback-Reflection: In contrast, feedback-reflection leverages external signals from user interaction or environment feedback to guide optimization. ECPO Feng et al. (2025) exemplifies this paradigm by modeling user expectations and confirmations at each conversational turn. It identifies sources of user dissatisfaction and performs fine-grained, turn-level preference updates. This design improves multi-turn interaction quality while avoiding the high sampling cost associated with prior methods such as MTPO.

4.3. Multi-Agent Recommenders (LoA L5)

Beyond the capabilities enabled by a single agent, multi-agent recommenders introduce an additional layer of enhancement by distributing decision-making across multiple specialized agents. 1) In terms of proactivity, multi-agent systems move from individual proactive reasoning to *collective proactivity*, where different agents—such as planners, critics, retrievers, and preference elicitors—jointly anticipate user needs and cross-check each other’s actions, leading to more reliable proactive behaviors than a single agent. 2) For context awareness, multi-agent architectures alleviate the limitations of a single agent’s context window by allowing each agent to maintain its own perspective or memory, resulting in distributed memory that captures user intent, item evidence, and environmental constraints more completely. 3) Regarding interaction flexibility, multi-agent setups transform single-agent multi-turn interaction into multi-party collaborative interaction: agents can debate, critique, refine, or negotiate with one another before presenting results to the user, enabling richer reasoning patterns than any single agent can achieve. 4) Finally, in terms of adaptivity, multi-agent systems support continuous evolving behavior not only through user feedback but also through internal self-correction loops—where agents iteratively refine each other’s outputs—making the system more adaptive and stable compared to a single adaptive policy. Overall, while single-agent systems enhance recommendation along the four dimensions within

one coherent policy, multi-agent systems amplify these capabilities through structured cooperation, role specialization, and self-consistency mechanisms.

4.3.1. Roles and Coordination Protocols

Common role sets include manager/planner agents, retrievers, rankers, analyzers, critics, verifiers, and safety monitors. Coordination protocols vary: manager-worker decomposition [Agarwal et al. \(2024\)](#); [Nie et al. \(2024\)](#); [Portugal et al. \(2024\)](#); [Wang et al. \(2024d\)](#); [Xia et al. \(2026\)](#); [Zhang et al. \(2026a\)](#), debate-judge selection [Fang et al. \(2024\)](#); [Ma et al. \(2025a\)](#), role-based trustworthy conversational protocols [Hui et al. \(2025\)](#), and negotiation-based approaches for multi-stakeholder recommendation [Banerjee et al. \(2025\)](#); [Dixit and Dixit \(2026\)](#). Hierarchical agent structures are also common in route and itinerary recommendation, where feasibility constraints naturally require staged reasoning and verification.

4.3.2. Communication

Communication is a key mechanism enabling collaboration and coordination in multi-agent recommendation systems. By exchanging information about user context, preferences, and reasoning outcomes, agents can jointly construct more accurate and consistent recommendations. Such inter-agent communication fosters cooperative behaviors, mitigates information asymmetry, and supports the emergence of specialized agent roles for complex recommendation tasks. MacRec [Wang et al. \(2024c\)](#) introduces a multi-agent collaboration framework, including the Manager, User/Item Analyst, Reflector, Searcher, and Task Interpreter, to enhance recommendation tasks. MAS4POI [Wu et al. \(2025b\)](#) is a multi-agent framework (comprising agents such as DataAgent, Manager, Analyst, and Navigator) that collaboratively produces next point-of-interest (POI) recommendations. In each task, one or more agents are selected to perform the corresponding operation, and all agents are coordinated and their information aggregated through the manager agent. MACRS [Fang et al. \(2024\)](#) decomposes the Conversational Recommender System (CRS) workflow into four LLM-based agents that plan dialogue acts, independently generate candidate responses, and then select a final response via a judge module. Beyond the policy side, MACRS also provides a more realistic user simulator by masking the target item with a keyword-level profile to reduce direct leakage and overly explicit hints. In ARAG [Maragheh et al. \(2025\)](#), agents communicate via staged hand-offs rather than free-form chat: understanding → alignment-scoring → evidence compression → ranking. This act-structured passing stabilizes dialogue flow and makes decisions diagnosable. In RecBot [Tang et al. \(2026\)](#), communication between the Parser and Planner agents follows a structured hand-off protocol, where the Parser conveys normalized user intents to the Planner through explicit JSON-style messages, ensuring transparent coordination and preventing semantic drift during multi-turn command execution. In Collab-REC [Banerjee et al. \(2025\)](#), communication proceeds via moderated rounds: agents submit act-conditioned proposals; the moderator feeds back rejections and penalties (e.g., repeated or spurious cities) and requests revisions, yielding negotiated consensus rather than free-form chat. TAIRA [Yu et al. \(2025a\)](#) coordinates a Manager-Executors pipeline via structured hand-offs: the Manager selects a thought pattern, decomposes the user request into sub-tasks, and dispatches them to specialized Executors, who return evidence and partial results for aggregation—reducing semantic drift compared with free-form multi-agent chat. In MATCHA [Hui et al. \(2025\)](#), inter-agent communication follows a structured role-based protocol, where the intent agent passes normalized user goals to the generator and ranker, and downstream explanation and safeguard agents exchange rationale and risk annotations to ensure coherent, transparent, and trustworthy collaboration.

4.3.3. Optimization

Optimization in multi-agent recommendation systems focuses on improving collective decision quality without relying on gradient-based parameter updates. Through mechanisms such as structured coordination, negotiation, and self-distillation, agents can iteratively refine their reasoning and cooperation strategies [Acharya and Mohbey \(2026\)](#); [Li et al. \(2026a\)](#); [Wang et al. \(2026a,b\)](#); [Wu et al. \(2026\)](#); [Zhu et al. \(2025c\)](#). Rather than relying on gradient-based updates, MACRS [Fang et al. \(2024\)](#) mainly optimizes the decision quality through structured coordination (parallel candidate generation followed by selection) and by adopting a more realistic user-simulation protocol. ARAG [Maragheh et al. \(2025\)](#) optimizes recommendation quality by coordinating agents for consistency-aware evidence refinement, enabling the system to iteratively filter irrelevant retrievals and enhance ranking accuracy without retraining the backbone LLM. RecBot [Tang et al. \(2026\)](#) optimizes its dual-agent policy through simulation-driven distillation, where high-fidelity trajectories from a teacher model (e.g., GPT-4.1) are distilled into a lightweight student agent, enabling efficient online deployment while preserving reasoning and planning competence. Collab-REC [Banerjee et al. \(2025\)](#) optimizes decision quality without fine-tuning LLMs by combining multi-round negotiation with a penalty-aware scoring function that rewards agent success while penalizing repetition and hallucinated entries—explicitly steering the system toward balanced, diverse lists across stakeholders. TAIRA [Yu et al. \(2025a\)](#) optimizes agent behavior through thought-pattern distillation, a self-improving process that abstracts successful reasoning traces into reusable planning templates, enabling more consistent task decomposition and tool use without retraining the underlying LLM.

4.4. Summary

Agent-as-recommender systems represent the most direct realization of autonomy in recommendation: the agent becomes the controller and the decision maker. The literature indicates a progression from single-agent interactive controllers, to tool-learning and reflective workflows, and finally to multi-agent orchestration for robustness, negotiation, and complex domains. From agentic augmentation (*i.e.*, agent-assisted recommender) to agentic replacement (*i.e.*, agent as recommender), recommender systems are more proactive, have stronger context awareness, are more flexible in interactions, and more adaptive (*cf.* §4.2 and §4.3).

5. Agentic Simulation: Data Synthesis, User Simulation, and Environment Simulation

In this survey, *simulation* refers to using one or more agents to approximate parts of the recommendation ecosystem that are expensive, unsafe, or impossible to observe directly. This includes generating synthetic training data, simulating user feedback to train or test recommenders, and simulating closed-loop environments where recommender policies interact with populations over time.

5.1. Why do we need simulation?

Across the literature, simulation is used for four recurring motivations: 1) **Training without expensive interaction.** Simulators provide behavioral signals when online exploration is infeasible (cold-start, privacy constraints, or limited traffic). 2) **Counterfactual and stress testing.** Simulators enable controlled tests of robustness, fairness, and safety properties that are difficult to measure from logs. 3) **Data synthesis and augmentation.** Synthetic dialogues, profiles, and feedback can reduce sparsity and broaden coverage. 4) **Ecosystem and feedback-loop analysis.** Multi-agent simulations can model long-term dynamics such as filter bubbles and echo chambers.

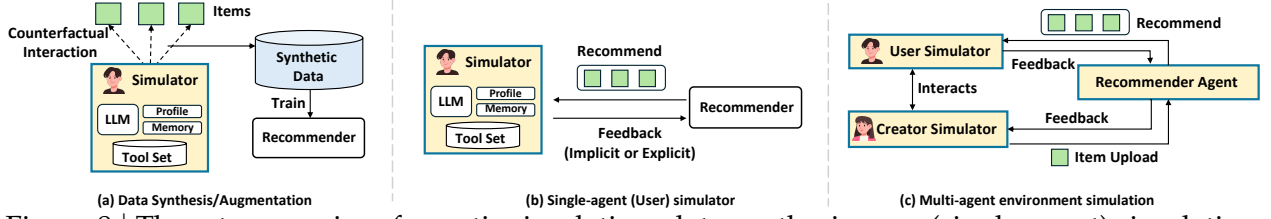


Figure 8 | Three taxonomies of agentic simulation: data synthesis, user (single-agent) simulation, and environment (multi-agent) simulation.

5.2. A taxonomy of simulation types

We propose a simple taxonomy that aligns with autonomy and the “what is being simulated” question.

5.2.1. (1) Data synthesis / data augmentation

Data synthesis uses agents to generate artifacts that support training or evaluation: synthetic conversations, item descriptions, preference rationales, or structured user profiles. TalkPlayData 2 exemplifies an agentic synthetic pipeline for multi-modal conversational music recommendation, demonstrating how agentic generation can create richer training signals than template-based augmentation [Choi et al. \(2025\)](#). For cold-start settings, simulator-style generation can replace missing interaction histories with plausible synthetic signals: a large language model simulator for cold-start recommendation explicitly targets the absence of real user interaction data [Huang et al. \(2025c\)](#). LAUS [Park \(2025\)](#) leverages large language models to simulate user interactions, enabling the training of news recommenders without relying on large-scale real user data. For reinforcement learning-based recommender systems, [Zhang et al. \(2025e\)](#) propose an interpretable user simulator that combines LLM-based preference reasoning with statistical behavior modeling to generate high-fidelity training data, thereby improving recommendation training.

5.2.2. (2) User simulation (single-agent simulators)

User simulation models user feedback (clicks, ratings, critiques, dialogue acts) conditioned on context. This can appear as a single LLM agent that plays the user role, or as structured simulators augmented by LLM reasoning. A key example is RecAgent, which frames simulation as a paradigm for recommender systems and highlights how LLM agents can generate interactive trajectories rather than static labels [Wang et al. \(2023a\)](#). User Behavior Simulation with LLM-based Agents provides a systematic formulation and is especially relevant because it positions LLM-agent simulators as a way to reproduce behavioral dynamics at scale [Wang et al. \(2025c\)](#), requiring alignment with real-world data distributions.

Design axes for user simulators. User simulators can be compared along: (i) **fidelity** (does the simulator match real distributions and causal responses?), (ii) **controllability** (can we steer demographics, noise, and drift?), (iii) **observability** (does the simulator output only implicit feedback or also explicit rationales?), and (iv) **calibration** (does the simulator’s actions reflect real behaviors?). These axes should be evaluated explicitly rather than assumed, because simulator errors can lead to “training on simulator artifacts.” In the following, we discuss the existing literature from these design axes, respectively.

(i) *Simulation fidelity.* To ensure high simulation *fidelity* aligned with real-world data distributions, user simulators primarily focus on two aspects: *profile construction* and *memory design*.

- **Profile construction.** Most existing user simulators construct personalized user profiles from real-world interaction data [Liu et al. \(2026\)](#); [Wanyan et al. \(2025\)](#). For example, RecAgent [Wang et al. \(2025b\)](#) and Lusifer [Ebrat et al. \(2025\)](#) build basic profiles using demographic and attribute information such as gender, age, traits, and interests, while Agent4Rec [Zhang et al. \(2024\)](#) derives social traits (e.g., activity, conformity, and diversity) from datasets like MovieLens-1M. In addition, several methods, including RecAgent [Wang et al. \(2023a\)](#) and Agent4Rec [Zhang et al. \(2024\)](#), leverage LLMs for generative profiling, such as summarizing representative behavioral roles, personalized preferences, and rating patterns.
- **Memory design.** Another key component to ensure fidelity is memory. It allows the agent to recall past actions, feedback, and outcomes, thereby supporting sequential decision-making and adaptive behavior in long-term recommendation simulations. Beyond saving long- and short-term interests, memory mechanisms in user simulators are designed to capture richer aspects of user behavior. For instance, RecAgent [Wang et al. \(2025b\)](#) introduces sensory memory, which directly interacts with the environment and transforms raw observations into concise representations. Agent4Rec [Zhang et al. \(2024\)](#) proposes factual memory, which encodes user interaction behaviors, and emotional memory, which captures the psychological states induced by these interactions.

(ii) *Simulator controllability.* A controllable user simulator adds an important dimension: *controllability* over behavior and dynamics, which is critical for evaluation and for testing robustness to demographic or preference shifts [Wei et al. \(2025c\)](#); [Zhu et al. \(2024, 2025a\)](#). [Zhu et al. \(2025a\)](#) proposes a plugin-based, controllable and human-in-the-loop LLM user simulator (CSHI), where controllability is enabled by a modular plugin manager for stage-wise behavior control, explicit user profile manipulation, and human intervention. Additionally, controllability can be achieved by adjusting the simulator’s memory decay rate to model interest drift [Wang et al. \(2023a\)](#); [Ye et al. \(2025\)](#), and by tuning the sampling distribution of user profiles to simulate different demographic compositions [Ye et al. \(2025\)](#); [Zhang et al. \(2024\)](#).

(iii) *Simulation observability.* Beyond outputting implicit feedback (e.g., like), improving *observability* of the recommendation process is another key ability of user simulator. [Zhang et al. \(2025e\)](#) propose an interpretable user simulator that integrates LLM-based preference reasoning with statistical behavior modeling to generate high-fidelity training data, thereby improving reinforcement learning-based recommender systems. For domain-specific simulators, LLM-as-user simulation for news recommendation targets training without real interactions [Park \(2025\)](#), while Lusifer provides an LLM-based simulated feedback environment for online recommender systems [Ebrat et al. \(2025\)](#).

(iv) *Behavior calibration.* Whether the simulator’s behavior reflects real user behavior is a critical aspect of simulator design. Existing research primarily models two types of actions: (1) user-system actions and (2) user-user actions.

- **User-system action.** In non-conversational user simulation, implicit signals (e.g., like, click) [Bougie et al. \(2026\)](#); [Huang et al. \(2025c\)](#); [Park \(2025\)](#); [Zhang et al. \(2024, 2025e\)](#) and explicit signals (e.g., comments, reviews) [Wang et al. \(2025b\)](#) constitute the two primary types of user-to-item actions in user simulation. To implicitly model, Agent4Rec [Zhang et al. \(2024\)](#) divides user page-by-page browsing actions into taste-driven and emotion-driven types: (1) taste-driven actions include viewing, rating, and expressing post-viewing feelings towards items, capturing users’ immediate preferences during page-by-page recommendations; (2) emotion-driven actions (i.e. existing the session or rating the recommender system) capture user affective states and reflect how satisfaction and fatigue shape their decision to continue

or quit. Beyond user feedback simulation, some works [Jin et al. \(2025\)](#); [Ye et al. \(2025\)](#) model item providers (e.g., creators or merchants), enabling simulators to update item attributes or generate new items to capture supply-side dynamics. In conversational recommender systems, users typically interact with the system through the generative capabilities of LLMs [Chen et al. \(2025b\)](#); [Feng et al. \(2025\)](#), generating fluent and contextually relevant responses to express preference.

- **User-user action.** To capture this broader social dimension, recent works [Wang et al. \(2025b\)](#) introduce conversational user simulators that equip LLM-based agents with the ability to interact with other user simulators, enabling more realistic and socially grounded simulations. RecAgent [Wang et al. \(2025b\)](#) not only enables users to perform conventional recommender system actions (e.g., searching, browsing), but also allows agents to engage in one-to-one chatting and one-to-many broadcasting.

5.2.3. (3) Closed-loop multi-agent environment simulation

The most ambitious form of simulation uses multiple agents to model populations and platform mechanisms, enabling closed-loop experiments over long horizons. RecoWorld provides simulated environments for agentic recommender systems, pushing toward standardized testbeds where policies can be compared under controlled dynamics [Liu et al. \(2025a\)](#). Filter bubble simulation with LLM agents demonstrates that such environments can capture emergent societal effects (e.g., self-reinforcing exposure patterns) that are central to multi-objective evaluation [Sukiennik et al. \(2025\)](#). GGBond extends this direction with a graph-based AI-agent society, highlighting the need to represent social structure and influence in ecosystem simulations [Zhong et al. \(2025\)](#). [Cai et al. \(2025b\)](#) proposes an agentic feedback loop framework that explicitly models the iterative interaction between a recommendation agent and a user agent, allowing both to co-evolve and improve through mutual feedback. Beyond user simulation, recent works extend the paradigm to multi-agent environments by modeling item providers (e.g., merchants, content creators). In this setting, simulators can act not only as a user model that mimics user behavior passively responding to recommendations, but also as an active recommendation agent that proactively generates and recommends items to other users. For example, CreAgent [Ye et al. \(2025\)](#) leverages LLMs to simulate a realistic content recommendation platform (i.e., YouTube) involving both users and content creators, enabling long-term offline evaluation of recommender systems with multi-stakeholders. RecInter [Jin et al. \(2025\)](#) introduces merchant agents that can respond to user feedback and dynamically modify item attributes, enabling a more realistic co-evolution of users, items, and providers.

5.3. Simulation shortcoming and open challenges

Despite rapid progress, simulation brings substantial risks: 1) **Simulator-policy mismatch.** Agents trained against a simulator may overfit to simulator quirks, leading to inflated offline performance and poor real-world transfer. This is particularly problematic for agentic systems that exploit loopholes in feedback generation. 2) **Compounding error in closed loops.** In closed-loop simulation, small misspecifications in user response can compound over time, producing misleading conclusions about long-term welfare. 3) **Evaluation of simulators themselves.** Simulators must be evaluated as *models*, not just used as tools. We argue that simulator evaluation should be treated as a first-class part of agentic recommender evaluation (Sec. 6), including calibration tests, distributional similarity tests, and targeted behavioral probes. 4) **Link to evaluation.** Simulation is simultaneously an *object* of evaluation and a *method* for evaluation; for agentic recommendation, the two are inseparable. We therefore treat simulation-based evaluation and red-teaming as core methodologies in the next section.

6. Evaluation and Benchmarking

Agentic recommender systems (ARS) should be evaluated as interactive systems rather than only as static rankers. In a conventional top- N setting, the main evaluation object is a ranked list compared with held-out interactions. In an ARS, the object is often a trajectory that includes user inputs, internal state, memory reads and writes, tool calls, observations, intermediate rationales, final outputs, and feedback. A system can rank relevant items while using unsupported evidence, exposing sensitive profile attributes, or making repeated tool calls; conversely, a fluent dialogue can still fail to recommend useful items. Evaluation therefore has to separate final recommendation utility from the process that produced it.

For an episode with T turns, we write the trajectory as

$$\tau = (u_1, s_1, a_1, o_1, y_1, \dots, u_T, s_T, a_T, o_T, y_T),$$

where u_t is the user input, s_t is the internal or environmental state, a_t is the agent action, o_t is an observation returned by a tool, memory, simulator, or another agent, and y_t is the user-facing output. Classical metrics mainly evaluate the ranked list contained in y_T . Agentic evaluation instead requires a vector of measurements,

$$m(\tau) = (m_{rec}, m_{int}, m_{gen}, m_{ground}, m_{tool}, m_{mem}, m_{safe}, m_{cost}),$$

where the components summarize recommendation utility, interaction quality, generated-output quality, grounding, tool and planning behavior, memory behavior, safety, and cost. The relevant components depend on both the role of the agent and its Level of Autonomy (LoA). L2 systems require retrieval and grounding checks; L3 systems require tool-use diagnostics; L4 systems require plan, memory, and reflection evaluation; and L5 systems require coordination and failure-attribution metrics. Likewise, agent-assisted recommenders evaluate the marginal value of agents around a classical pipeline, agent-as-recommenders evaluate end-to-end agent behavior, and agent-as-simulators evaluate both the simulator and the recommender policies tested with it.

6.1. Targets, Protocols, and Metrics

A recurring ambiguity in ARS evaluation is the conflation of targets, protocols, and metrics as summarized in Table 6. A *target* is the object being evaluated: the ranked list, conversation, explanation, RAG evidence, simulator, tool trace, memory update, multi-agent coordination process, or deployed service. A *protocol* is the procedure used to evaluate the target: offline benchmark, simulation, user study, human annotation, LLM-as-judge, adversarial testing, case study, or online experiment. A *metric* is the observable measurement produced by the protocol: NDCG, Recall, Success@K, average turns, BLEU, faithfulness, attack success rate, latency, token cost, or GMV.

The H0–H3 distinction also helps relate evaluation to LoA. At L2, the main additional target beyond ranking is retrieved evidence: whether user, item, or auxiliary knowledge is relevant and used faithfully. At L3, the target expands to tool traces: tool choice, argument validity, execution success, and recovery. At L4, the target includes plan quality, memory updates, and self-correction. At L5, the target includes communication, agreement, and contribution of each specialist agent. This level-wise view prevents a common mismatch in which a paper claims planning, memory, or multi-agent benefits but evaluates only the final ranked list.

Offline evaluation remains useful because it is cheap, reproducible, and compatible with existing recommender baselines. It is not, however, sufficient evidence of agentic behavior. A high NDCG or HR score does not show that the agent selected the right tool, grounded claims in catalog evidence,

Table 6 | Evaluation matrix for agentic recommender systems.

H0 target	H1 protocols	H2 purpose	H3 metrics and required context
Recommendation outcome	Offline benchmark; simulation; online or A/B test.	Test whether the final item, list, route, or POI set is useful.	HR/Hit, Recall, Precision, NDCG, MRR, MAP, AUC, CTR/CVR. Report split, candidate set, negative sampling, baselines, LLM/backbone version, and repeated runs.
Interaction and generated output	CRS benchmark; user study; dialogue simulation; human or LLM judge; reference comparison.	Test preference elicitation, task completion, response quality, and explanation usefulness.	Success@K, task completion, average turns, satisfaction, BLEU/ROUGE/METEOR/BERTScore, clarity, relevance, helpfulness. Report rubric, judge prompt, evidence visibility, and agreement.
Grounding and RAG evidence	Retrieval benchmark; claim-level audit; noisy or adversarial evidence test.	Test whether retrieved user, item, or auxiliary evidence supports the recommendation and explanation.	Retrieval Recall@K/NDCG, context relevance, citation accuracy, faithfulness, hallucination and contradiction rate. Report source, granularity, freshness, retrieval budget, and claim segmentation.
Simulator validity	Simulator-vs-log comparison; human validation; downstream utility; simulation-to-A/B alignment.	Test whether simulated users, creators, or environments are plausible and useful.	Distributional similarity, rating/click consistency, diversity, leakage rate, reward, liking ratio, treatment-effect correlation. Report calibration data, persona construction, leakage controls, and uncertainty.
Agent trace: tools, memory, planning, coordination	Trace logging; functional validation; module ablation; error taxonomy; case study.	Attribute success and failure to agentic components, not only to the final list.	Tool-selection accuracy, argument validity, execution success, plan coverage, recovery success, memory precision/recall, contradiction rate, agreement, time-to-consensus. Report tool schema, memory store, trace format, and agent-role ablations.
Safety, privacy, fairness, and deployment	Red teaming; poisoning/jailbreak tests; privacy audit; fairness analysis; efficiency measurement; production monitoring.	Test robustness, policy consistency, privacy preservation, fairness, and operational feasibility.	Attack success, policy violation, privacy leakage, exposure imbalance, diversity, latency, token/inference cost, throughput, timeout/fallback rate, retention/GMV. Report threat model, attack budget, protected/provider groups, hardware/API setup, traffic split, and confidence intervals.

maintained a consistent profile, asked useful clarifying questions, or satisfied latency constraints. Simulation, user studies, LLM-as-judge protocols, adversarial tests, and online experiments answer different questions and should be treated as complementary. Simulation tests controlled interaction; user studies test perceived utility; LLM judges scale qualitative assessment but require calibration; adversarial evaluation probes robustness; and online experiments provide causal evidence under deployment constraints.

6.2. Recommendation, Conversation, and Generated Output

Classical recommender metrics remain the foundation when the target is a final item choice or ranked list. Most agent-assisted and agent-as-recommender papers therefore report HR, Recall, Precision, NDCG, MRR, MAP, AUC, or CTR-like outcomes. InteRecAgent, ToolRec, AgentDR, MADREC, AgenticRAG, R4EC, and VRAgent-R1 retain top- K or ranking metrics to show that agentic workflows do not reduce final-list relevance [Chen et al. \(2025d\)](#); [Gu et al. \(2025\)](#); [Huang et al. \(2025d\)](#); [Ma et al. \(2025b\)](#); [Park and Kim \(2025\)](#); [Yang et al. \(2025a\)](#); [Zhao et al. \(2024a\)](#). These metrics should be interpreted with ablations. In L2 systems, gains may arise from better retrieval or prompt construction; in L3 systems, they may hide brittle tool routing; in L4 systems, they may depend on reflection prompts; and in L5 systems, they may reflect one useful specialist rather than effective coordination. Reporting should therefore include the candidate set, negative sampling

protocol, split, baselines, LLM version, number of tool calls, memory size, reflection rounds, and repeated-run variance.

Cold-start and sparse-data settings illustrate why classical metrics remain important. In these settings, agentic components may supply missing preference evidence, convert natural-language constraints into structured signals, or generate synthetic interactions. Ranking metrics can measure whether these additions improve downstream recommendation utility. They do not, however, establish why the improvement occurs.

Conversational ARS add an interaction layer between intent and recommendation. Evaluation must measure not only whether a target item appears, but also whether the agent elicits missing preferences, follows commands, adapts to feedback, remains coherent across turns, and stops at an appropriate time. Common metrics include success rate, Success@K, average turns, task completion, satisfaction, engagement, and willingness to reuse. MACRS and ECPO are representative because they evaluate multi-turn conversational recommendation and expectation confirmation rather than only static ranking [Fang et al. \(2024\)](#); [Feng et al. \(2025\)](#). RecBot, SmartEats, WeMusic-Agent, and AdaptJobRec further illustrate settings where command following, dietary constraints, music interaction, or job-preference elicitation make user experience part of the recommendation target [Haller et al. \(2026\)](#); [Liang et al. \(2025\)](#); [Tang et al. \(2026\)](#); [Wang et al. \(2025e\)](#).

Explanations and other generated outputs require separate evaluation. A rationale can be fluent but unsupported, concise but unhelpful, or faithful to an item attribute but irrelevant to the user’s current constraint. MADREC and REXHA evaluate explanation quality, CARTS focuses on recommendation textual summarization, and AgenticRAG combines recommendation with retrieval-grounded explanation [Chen et al. \(2025a\)](#); [Ma et al. \(2025b\)](#); [Park and Kim \(2025\)](#); [Sun et al. \(2025\)](#). Automatic text-overlap metrics such as BLEU, ROUGE, METEOR, and BERTScore are useful when references exist, but they do not establish factual support or user usefulness. Human or LLM judges should therefore score at least two axes: communicative quality (clarity, relevance, naturalness, personalization, usefulness) and evidence faithfulness (whether factual claims are supported by item metadata, user history, reviews, knowledge-base triples, or retrieved passages). Judge prompts, rubrics, visibility of evidence, identity masking, item-order randomization, and agreement statistics should be reported.

6.3. Grounded Retrieval and RAG Evaluation

Retrieval-augmented generation is common in ARS because recommendation often depends on information outside model parameters: user profiles and histories, item attributes, reviews, knowledge graphs, web pages, policy rules, and domain constraints. In the LoA taxonomy, this is central to L2 systems and becomes a component of L3–L5 systems when retrieval is embedded in tools, planning, or multi-agent workflows. RAG evaluation should distinguish retrieval quality from grounded generation. A recommender can retrieve relevant items but generate an unsupported explanation, or retrieve weak evidence and still guess a plausible item.

The evidence space has three sources: *user information*, *item information*, and *auxiliary knowledge*. RA-Rec, CORAL, ARAG, and AgenticRAG illustrate retrieval-augmented recommendation and explanation settings where evidence quality should be checked separately from final ranking [Kemper et al. \(2024\)](#); [Ma et al. \(2025b\)](#); [Maragheh et al. \(2025\)](#); [Wu et al. \(2024a\)](#). 1) **Retriever diagnostics** should include Recall@K, Precision@K, MRR, NDCG, coverage of required attributes, diversity of evidence sources, and freshness when data changes over time. 2) **Generator diagnostics** should include answer relevance, claim-level citation accuracy, faithfulness, hallucination rate, contradiction rate, and consistency with the dialogue context. 3) **Robustness tests** should include

insufficient evidence, conflicting evidence, poisoned item descriptions, and unsupported user requests.

RAG evaluation should also report how the retrieved context is used. Merely retrieving a relevant review or item attribute does not guarantee that the generator used it correctly. Claim-level checking is preferable for user-facing rationales: each factual statement should be linked to a supporting user profile entry, item attribute, review, knowledge-graph fact, policy rule, or web/API result. When the system compares alternatives, the comparison should be checked against the same candidate set and the same time-sensitive values, such as price, availability, or distance. Negative controls are also useful: if evidence is removed or contradicted, the agent should either abstain, ask a clarification, or revise the claim rather than generate unsupported text.

6.4. Simulator Evaluation

Simulation is both a method for evaluating ARS and an object that must be evaluated [Yoon et al. \(2024\)](#); [Zhang et al. \(2026b\)](#). A simulator that improves downstream NDCG can still misrepresent real users, reveal labels too directly, collapse behavioral diversity, or reward policies that exploit simulator artifacts. Simulator evaluation should therefore distinguish *downstream utility* from *simulator validity*. 1) **Downstream utility** asks whether simulated interactions improve a recommender under HR, NDCG, Recall, reward, liking ratio, or inference-time metrics. Validity asks whether simulated users, creators, and environments behave like the intended population. 2) **Validity checks** should include distributional fidelity to logs, behavioral realism under persona constraints, diversity across personas, leakage of target items or labels, and calibration against human judgments or online treatment effects. Long-term environments should additionally report preference drift, novelty decay, provider exposure, diversity, retention, and feedback-loop effects. Without such checks, simulation may be useful for development but weak as evidence of real-world behavior.

A useful simulator report should make the validity conditions explicit. If the simulator is calibrated on a particular domain, interaction type, or user population, the paper should not imply that its conclusions transfer automatically to other domains. It should also state what the simulator observes. A simulator with access to hidden target labels, full item metadata, or future interactions can produce overly optimistic results. A simulator that produces both implicit feedback and rationales should evaluate the two outputs separately, because plausible text does not imply realistic click or rating behavior. For long-horizon environments, the report should include stability checks over repeated interactions; otherwise small response-model errors may compound into misleading claims about retention, fairness, or filter bubbles.

6.5. Trace, Deployment, and Trustworthiness Evaluation

Agentic systems expose intermediate traces that should be evaluated directly. 1) For tool-driven agents, the trace should show whether the agent select the correct tool, supplied valid arguments, interpret the result correctly, recover from tool failures, and avoid unnecessary calls. ToolRec, RecMind, iAgent, AgentDR, and TAIRA motivate such diagnostics because their claimed contribution depends on explicit tool use, planning, shielding, or thought-pattern selection [Wang et al. \(2024a\)](#); [Xu et al. \(2025b\)](#); [Yang et al. \(2025a\)](#); [Yu et al. \(2025a\)](#); [Zhao et al. \(2024a\)](#). For memory, evaluation should measure retrieval precision and recall, update faithfulness, stale-memory use, contradiction rate, and privacy leakage. 2) For planning, evaluation should check task decomposition, constraint coverage, step ordering, verification, and recovery. For multi-agent systems such as MACRS, MACRec, Collab-REC, MAP, and MATCHA, evaluation should include agreement, vote entropy, number of rounds, time-to-consensus, role ablations, deadlock frequency,

and the marginal utility of each specialist agent [Banerjee et al. \(2025\)](#); [Fang et al. \(2024\)](#); [Hui et al. \(2025\)](#); [Lee et al. \(2025\)](#); [Wang et al. \(2024d\)](#).

Deployment evaluation is where many agentic designs become difficult to justify. Customer-facing ARS need latency, token cost, inference cost, throughput, tool-call count, cache-hit rate, timeout rate, fallback rate, and reliability metrics, in addition to online outcomes such as CTR, CVR, retention, satisfaction, or GMV. ColdLLM is an example of combining offline cold-start metrics with online-style business evidence, while efficiency-aware RAG and cloud-device collaborative recommendation show that accuracy must be studied jointly with latency, cost, and system boundaries [Huang et al. \(2025c\)](#); [Long et al. \(2025\)](#); [Zhou et al. \(2025\)](#). The most informative reports present trade-offs, such as NDCG versus latency or success rate versus tokens per episode, rather than a single best score. Parallelism should also be reported, since multi-agent systems may reduce wall-clock time while increasing total token cost and orchestration complexity.

Deployment metrics should be reported with the same care as ranking metrics. Mean latency is insufficient without tail latency, since multi-step agents may fail through long-tail tool delays. Token cost should distinguish prompt, retrieval, reasoning, verifier, and explanation stages where possible. Tool-call count should be reported per episode and, for multi-agent systems, per role. Fallback behavior should also be measured: a system that silently falls back to a generic ranker after tool failure may appear reliable but no longer exercises the agentic workflow being studied. Online experiments should state traffic split, duration, confidence intervals, guardrail metrics, and whether users were exposed to generated explanations or only to final items.

Trustworthiness should be a primary evaluation target. RAG exposes item descriptions and knowledge bases to poisoning; memory can be corrupted, stale, or privacy-sensitive; tool use can trigger unsafe API calls; and multi-agent communication can propagate inconsistent state. Robustness protocols should include adversarial item descriptions, shilling or poisoning attacks, prompt injection, memory perturbation, and corrupted retrieval evidence. The poisoning study on retrieval-augmented recommenders and the DrunkAgent memory-perturbation work illustrate these risks [Gu et al. \(2026\)](#); [Li et al. \(2026b\)](#); [Nazary et al. \(2025\)](#); [Ning et al. \(2024\)](#); [Yang et al. \(2025b\)](#). Privacy evaluation should measure what is stored, where it is stored, how it is retrieved, whether it can be deleted, and whether sensitive attributes are exposed to other agents or tools [Long et al. \(2025\)](#); [Zhang et al. \(2025b\)](#). Fairness and diversity evaluation should consider users, items, creators, and providers; multi-stakeholder recommendation and long-term simulation show that exposure and diversity effects may appear only after repeated interaction [Banerjee et al. \(2025\)](#); [Jannach et al. \(2025\)](#); [Sukiennik et al. \(2025\)](#); [Ye et al. \(2025\)](#).

A practical evaluation stack should therefore be layered. Offline metrics provide fast regression tests for relevance. Trace metrics diagnose tools, memory, plans, and coordination. Grounding metrics audit factual support. Simulator metrics test controlled interaction and long-horizon effects, subject to calibration. Human or LLM judges assess aspects of communication that are difficult to reduce to item relevance. Deployment metrics test whether the design is feasible under latency, cost, and reliability constraints. Online experiments, when available, provide the strongest evidence for user and business impact. No single protocol is sufficient for all ARS claims.

The checklist should be read as a minimum rather than a complete benchmark specification. Different applications may need additional domain-specific measurements. A travel planner may require route feasibility and weather or availability checks; a food recommender may require dietary and allergen constraints; a learning-path recommender may require prerequisite consistency; and a marketplace recommender may require provider exposure and inventory constraints. These domain-specific checks should be attached to the relevant target in Table 6 rather than introduced

Table 7 | Minimum reporting checklist for ARS evaluation.

Evaluation component	Minimum information to report
Offline recommendation	Data split, candidate set, negative sampling, baselines, repeated runs, LLM/backbone version.
Conversation and generation	User or simulator protocol, task definition, stopping criteria, judge rubric, prompt, evidence visibility, agreement statistics.
RAG and grounding	Retrieval source, evidence granularity, freshness, retrieval budget, claim segmentation, faithfulness or contradiction checks.
Tools and planning	Tool schema, tool-call logs, argument-validity checks, execution success, recovery policy, planner or verifier ablation.
Memory	Memory type, update rule, retrieval budget, stale-memory policy, contradiction checks, privacy and deletion handling.
Multi-agent coordination	Agent roles, message format, number of rounds, consensus rule, deadlock handling, role ablations, cost per role.
Safety and privacy	Threat model, attack budget, policy test suite, defense baselines, sensitive attributes, leakage metric.
Deployment	Hardware/API setup, latency distribution, token and inference cost, throughput, timeout/fallback rates, traffic split, confidence intervals.

as isolated metrics. This keeps the evaluation design interpretable: the reader can see which system claim is being tested, which protocol supports it, and which metric is used to report it.

6.6. Open Evaluation Gaps

The main evaluation gap is that architecture has advanced faster than measurement. Many papers introduce memory, tools, RAG, planning, reflection, or multiple agents, but still evaluate mainly with static ranking metrics. Future benchmarks should record full trajectories and compute layered metrics for recommendation utility, interaction quality, generated-output quality, grounding, tool correctness, memory behavior, safety, and cost. Calibrated simulation is another priority: simulator papers should report distributional fidelity, behavioral probes, leakage controls, diversity, uncertainty, and correlation with human or online outcomes. Finally, multi-agent evaluation should make coordination measurable through message-level diagnostics, specialist-agent ablations, contribution analysis, and cost-latency-quality trade-offs. These practices would make it clearer when agentic components improve recommendation and when a simpler agent-assisted or conventional pipeline is sufficient.

7. Open Challenges and Opportunities

Agentic recommender systems introduce new capabilities beyond traditional recommendation pipelines, yet they also expose several unresolved challenges. We organize the open problems and opportunities from three complementary perspectives: the modeling perspective (Section 7.1), the user perspective (Section 7.2), and the system perspective (Section 7.3).

7.1. Modeling Perspective

7.1.1. Lifelong User Modeling

A fundamental challenge lies in capturing a user’s evolving interests across long time horizons. Existing agentic systems often treat user representations as short-term context, which limits

their ability to retain stable preferences while adapting to new signals. Agents may overfit to recent interactions or lose information about inactive users. An opportunity is to design lifelong memory mechanisms that distinguish persistent preferences from transient behaviors. This can be operationalized through hierarchical memory blocks, user-specific episodic buffers, or retrieval-augmented preference summaries. Another direction is to define a new evaluation task on lifelong preference retention, allowing systematic assessment of how well an agent maintains user identity across continual updates.

7.1.2. Contextual Modeling

Recommendation relevance depends strongly on context, yet most current agent workflows do not differentiate between short-term triggers, mid-term sessions, and long-term routines. Agents often rely on raw history, which mixes signals of varying importance and introduces noise. A promising opportunity is to adopt context engineering principles that help the agent identify, compress, and structure contextual information before reasoning. This includes context abstraction modules that re-write user history into semantically meaningful summaries, as well as context-selection tasks where the agent must choose an optimal subset of history for the downstream recommendation. Such mechanisms encourage structured reasoning and reduce reliance on long unfiltered sequences.

7.1.3. Multimodal Modeling

Modern recommendation scenarios incorporate text, images, video, and audio. Agents face two major challenges: building unified multimodal representations and aligning these signals with user preferences. Naïve fusion often leads to inconsistent semantics and weak cross-modal grounding. Opportunities include designing multimodal tokenization schemes that convert heterogeneous content into a shared token space, enabling smoother reasoning. Cross-modal alignment modules can be integrated into the agent workflow to enforce consistency between modalities. In addition, multimodal reasoning tasks can be formulated to benchmark how well the agent interprets visual and textual cues jointly. These tasks can guide the development of agents that can explain recommendations grounded in both semantics and appearance.

7.2. User Perspective

7.2.1. Controllability

Users often lack clarity on how an agent arrives at a decision. Current agentic systems offer limited control over the decision process or the ability to steer recommendations through explicit feedback. Opportunities arise in designing interfaces and agent workflows that expose intermediate decisions. Decision-process visibility can be supported by traceable reasoning graphs. Feedback-based control modules can allow users to adjust preference dimensions or veto specific reasoning paths. This improves both personalization and system transparency.

7.2.2. Explainability and Trustworthiness

Agentic recommender systems rely on multi-step reasoning, but these chains may introduce errors, hallucinations, or unsupported claims. Users may find it difficult to trust an opaque reasoning process. Agents can incorporate self-verification modules that check consistency across reasoning steps. Structured explanation templates can enforce factual grounding and avoid speculative statements. Defining trustworthiness benchmarks for agentic recommendation—for instance,

accuracy of reasoning chains or stability across perturbations—would help standardize evaluation and guide future development.

7.2.3. Privacy Preservation

Agents operate on large contextual windows and may inadvertently expose sensitive information during reasoning or tool use. Maintaining user privacy across memory, retrieval, and agent-to-agent communication is challenging. Opportunities include privacy-aware memory architectures that control what information is stored or exposed, and fine-grained permission protocols that regulate tool usage. Developing privacy-preserving reasoning tasks would further encourage models to separate preference inference from identifiable personal details.

7.3. System Perspective

7.3.1. Scalability

Agentic systems often require multiple agents or repeated workflow iterations to complete a task, which leads to scalability issues in large-scale recommendation environments. It remains unclear when specialized agents are necessary or how to coordinate multi-agent collaboration. A solution is to introduce agent orchestration frameworks that determine when to call which agent, based on task complexity or user state. Designing adaptive workflow planners that limit unnecessary reasoning cycles can further improve scalability. A benchmark for multi-agent recommendation settings would also help identify the optimal structure for different scenarios.

7.3.2. Efficiency in Training and Inference

Training and serving agentic models can be computationally expensive due to long context windows, multi-step reasoning, and external tool calls. Inference latencies may restrict real-world adoption. Opportunities lie in designing efficient training paradigms, such as distilling multi-step reasoning into shorter latent plans, storing reusable intermediate states, or learning compact preference embeddings. For inference, caching partial reasoning results and developing fast verification modules can significantly reduce latency. Clear system-level metrics that capture efficiency–performance trade-offs will guide deployment decisions.

7.3.3. API and Interaction-Level Efficiency

Many agentic systems rely on external APIs for search, item retrieval, or environment interaction. Excessive calls can bottleneck performance and increase costs. One opportunity is to develop local proxy tools or lightweight internal knowledge stores that reduce dependency on external APIs. Another is to design agent–tool protocols that batch operations or estimate when tool calls are unnecessary. These improvements can stabilize real-time recommendation workloads.

8. Conclusions

Agentic recommender systems represent a significant shift from passive item-ranking pipelines to autonomous, goal-directed systems capable of reasoning, planning, acting, and adapting within complex environments. By integrating large language model agents with profiles, memory structures, tool interfaces, and multi-step workflows, modern systems move beyond traditional assumptions about static user intents and fixed recommendation stages. Our survey organizes this emerging field through three complementary paradigms, *i.e.*, agent-assisted recommendation,

agent-as-recommender, and agent-as-user-simulator. This survey situates them within a unified autonomy framework that spans retrieval augmentation, tool use, single-agent planning, and multi-agent collaboration.

Across studies, we observe clear trends. Agent-assisted systems demonstrate strong gains in semantic understanding, retrieval quality, and contextual relevance. Single-agent recommenders further introduce coherent reasoning, long-horizon planning, and dynamic preference modeling, while multi-agent frameworks enable division of labor, critique, negotiation, and consensus building. User simulation agents provide new opportunities for scalable evaluation, counterfactual analysis, and environment modeling, allowing the field to explore settings that are difficult to observe in real-world logs. Despite these advances, substantial challenges remain. Lifelong user modeling, contextual abstraction, and multimodal grounding are still far from solved. User experience issues such as controllability, explainability, trustworthiness, and privacy require dedicated attention as systems become more autonomous. At the system level, scalability, efficiency, and robust agent-tool orchestration will determine whether agentic approaches can be deployed at industrial scale. Addressing these challenges creates opportunities to propose new tasks, design specialized modules, and introduce capabilities that reshape the user experience of recommendation.

References

- M. Acharya and K. K. Mohbey. Group poi recommendations optimization in multi-agent context-aware environment. *World Wide Web*, 29(1):9, 2026.
- P. Agarwal, H. Dave, J. Bandlamudi, R. Sindhgatta, and K. Mukherjee. Multi-stage prompting for next best agent recommendations in adaptive workflows. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 2024.
- F. S. Alamri, A. Rehman, B. Alghofaily, A. Ahmed, and K. Saleem. Leveraging rag with transformer for context based personalized recommendations. *IEEE Access*, 2025.
- X. Ao, S. Han, Y. Li, H. Ma, P. Zhang, and J. Zou. Retrieval augmented multi-agent recommender. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 2968–2972, 2025.
- A. Balachandran and M. Masum. Visiorag: A multimodal framework for enhancing recommendation system using vision transformers and rag. In *2025 IEEE Conference on Artificial Intelligence (CAI)*, pages 114–119. IEEE, 2025.
- A. Banerjee, A. Satish, and W. Wörndl. Enhancing tourism recommender systems for sustainable city trips using retrieval-augmented generation. In *International Workshop on Recommender Systems for Sustainability and Social Good*, pages 19–34. Springer, 2024.
- A. Banerjee, A. Satish, F. N. Aisyah, W. Wörndl, and Y. Deldjoo. Collab-rec: An llm-based agentic framework for balancing recommendations in tourism. *arXiv preprint arXiv:2508.15030*, 2025.
- G. Bhattacharya, B. Vivek, P. Bhargav, J. Gubbi, B. V., and A. Pal. Recommendation by generation: Generation augmented complementary fashion item retrieval using incomplete outfit. In *2025 IEEE/CVF International Conference on Computer Vision Workshops, ICCVW 2025*, 2025.
- W. Bi, Y. Mao, X. Liu, K. Tian, J. Zhang, H. Wang, and W. Que. Wemusic-agent: Efficient conversational music recommendation via knowledge internalization and agentic boundary learning. *arXiv preprint arXiv:2512.16108*, 2025.

- S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. B. Van Den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, et al. Improving language models by retrieving from trillions of tokens. In *ICML*, pages 2206–2240. PMLR, 2022.
- N. Bougie, G. M. Marconi, T. Yip, and N. Watanabe. Alignuser: Human-aligned llm agents via world models for recommender system evaluation. *arXiv preprint arXiv:2601.00930*, 2026.
- H. Cai, Y. Li, W. Wang, F. Zhu, X. Shen, W. Li, and T.-S. Chua. Large language models empowered personalized web agents. In *Proceedings of the ACM on Web Conference 2025*, pages 198–215, 2025a.
- S. Cai, J. Zhang, K. Bao, C. Gao, Q. Wang, F. Feng, and X. He. Agentic feedback loop modeling improves recommendation and user simulation. In *Proceedings of the 48th International ACM SIGIR conference on Research and Development in Information Retrieval*, pages 2235–2244, 2025b.
- T. Carraro, B. Singh, and N. Pedanekar. Large language model-based recommendation system agents. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*, pages 1340–1342, 2025.
- J. Chen, K. Yao, R. Y. Maragheh, K. Zhao, J. Xu, J. Cho, E. Korpeoglu, S. Kumar, and K. Achan. Carts: Collaborative agents for recommendation textual summarization. *arXiv:2506.17765*, 2025a.
- L. Chen, Q. Dai, Z. Zhang, X. Feng, M. Zhang, P. Tang, X. Chen, Y. Zhu, and Z. Dong. Recusersim: A realistic and diverse user simulator for evaluating conversational recommender systems. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 133–142, 2025b.
- L. Chen, W. Yuan, T. Chen, X. Zhao, N. Q. V. Hung, and H. Yin. Multi-agents based user values mining for recommendation. *arXiv preprint arXiv:2505.00981*, 2025c.
- S. Chen, B. Chen, C. Yu, Y. Luo, O. Yi, L. Cheng, C. Zhuo, Z. Li, and Y. Wang. Vragent-r1: Boosting video recommendation with mllm-based agents via reinforcement learning. *arXiv preprint arXiv:2507.02626*, 2025d.
- W. Chen, Y. Zhao, J. Huang, Z. Ye, C. M. Ju, T. Zhao, N. Shah, L. Chen, and Y. Zhang. Memrec: Collaborative memory-augmented agentic recommender system. *arXiv preprint arXiv:2601.08816*, 2026.
- X. Cheng, Z. Zhang, J. Wang, L. Fang, C. He, Q. Guan, S. Pan, and W. Luo. Education-oriented graph retrieval-augmented generation for learning path recommendation. *arXiv e-prints*, pages arXiv–2506, 2025.
- S. H. Cho, Y. Yang, D. Baeck, M. Kim, Y.-M. Kim, H. Lee, and S. Park. MARC: Multimodal and multi-task agentic retrieval-augmented generation for cold-start recommender system. *CoRR*, abs/2511.08181, 2025.
- K. Choi, S. Doh, and J. Nam. Talkplaydata 2: An agentic synthetic data pipeline for multimodal conversational music recommendation. *arXiv:2509.09685*, 2025.
- D. de Mijolla, W. Yang, P. Duckett, C. Frye, and M. Worrall. Language hooks: a modular framework for augmenting llm reasoning that decouples tool usage from the model and its prompt. *arXiv preprint arXiv:2412.05967*, 2024.
- Y. Deldjoo, Z. He, J. McAuley, A. Korikov, S. Sanner, A. Ramisa, R. Vidal, M. Sathiamoorthy, A. Kasirzadeh, and S. Milano. A review of modern recommender systems using generative models (gen-recsys). In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 6448–6458, 2024a.

- Y. Deldjoo, Z. He, J. McAuley, A. Korikov, S. Sanner, A. Ramisa, R. Vidal, M. Sathiamoorthy, A. Kasrizadeh, S. Milano, et al. Recommendation with generative models. *arXiv preprint arXiv:2409.15173*, 2024b.
- A. Dixit and S. Dixit. Pcn-rec: Agentic proof-carrying negotiation for reliable governance-constrained recommendation. *arXiv preprint arXiv:2601.09771*, 2026.
- D. Ebrat, E. Paradalis, and L. Rueda. Lusifer: Llm-based user simulated feedback environment for online recommender systems. In *2025 IEEE 4th International Conference on Computing and Machine Intelligence (ICMI)*, pages 1–6. IEEE, 2025.
- J. Fang, S. Gao, P. Ren, X. Chen, S. Verberne, and Z. Ren. A multi-agent conversational recommender system. *arXiv preprint arXiv:2402.01135*, 2024.
- X. Feng, J. Zhang, J. Tang, W. Li, G. Cai, X. Chen, Q. Dai, Y. Zhu, and Z. Dong. Expectation confirmation preference optimization for multi-turn conversational recommendation agent. In *ACL*, pages 5896–5914, 2025.
- H. Gu, R. Zhong, Y. Xia, W. Yang, C. Lu, P. Jiang, and K. Gai. R 4ec: A reasoning, reflection, and refinement framework for recommendation systems. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*, pages 411–421, 2025.
- S. Gu, J. Liu, D. Li, G. Zhang, M. Han, H. Gu, P. Zhang, N. Gu, L. Shang, and T. Lu. Llm agent-based shilling attack on recommender systems. In *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining*, pages 163–173, 2026.
- T. Guo, C. Liu, H. Wang, V. Mannam, F. Wang, X. Chen, X. Zhang, and C. K. Reddy. Knowledge graph enhanced language agents for recommendation. *arXiv:2410.19627*, 2024.
- Z. Guo, T. Tu, Y. Ma, and X. Yang. Vibemus: Proactive agentic system for music personalization. In *Proceedings of the 7th ACM International Conference on Multimedia in Asia*, pages 1–3, 2025.
- O. Haller, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai. Impress: Designing and evaluating a lightweight implicit recommender system in conversational support agents. In *Proceedings of the 31st International Conference on Intelligent User Interfaces*, pages 156–173, 2026.
- J. Hu, T. Wang, B. Yang, and H. Wang. Cogrec: A cognitive recommender agent fusing large language models and soar for explainable recommendation. *arXiv preprint arXiv:2512.24113*, 2025.
- C. Huang, H. Huang, T. Yu, K. Xie, J. Wu, S. Zhang, J. Mcauley, D. Jannach, and L. Yao. A survey of foundation model-powered recommender systems: From feature-based, generative to agentic paradigms. *arXiv preprint arXiv:2504.16420*, 2025a.
- C. Huang, J. Wu, Y. Xia, Z. Yu, R. Wang, T. Yu, R. Zhang, R. A. Rossi, B. Kveton, D. Zhou, et al. Towards agentic recommender systems in the era of multimodal large language models. *arXiv preprint arXiv:2503.16734*, 2025b.
- F. Huang, Y. Bei, Z. Yang, J. Jiang, H. Chen, Q. Shen, S. Wang, F. Karray, and P. S. Yu. Large language model simulator for cold-start recommendation. In *WSDM*, pages 261–270, 2025c.
- X. Huang, J. Lian, Y. Lei, J. Yao, D. Lian, and X. Xie. Recommender ai agent: Integrating large language models for interactive recommendations, 2024. URL <https://arxiv.org/abs/2308.16505>.

- X. Huang, J. Lian, Y. Lei, J. Yao, D. Lian, and X. Xie. Recommender ai agent: Integrating large language models for interactive recommendations. *TOIS*, 43(4):1–33, 2025d.
- Z. Hui, X. Wei, Y. Jiang, K. Gao, C. Wang, F. Ong, S.-e. Yoon, R. Pareek, and M. Gong. Matcha: Can multi-agent collaboration build a trustworthy conversational recommender? *arXiv preprint arXiv:2504.20094*, 2025.
- D. Jannach, A. Delić, F. Ricci, and M. Zanker. Rethinking group recommender systems in the era of generative ai: From one-shot recommendations to agentic group decision support. *arXiv preprint arXiv:2507.00535*, 2025.
- S. Jin, J. Zhang, Y. Liu, X. Zhang, Y. Zhang, G. Yin, F. Jiang, W. Lin, and R. Yan. Beyond static testbeds: An interaction-centric agent simulation platform for dynamic recommender systems, 2025. URL <https://arxiv.org/abs/2505.16429>.
- S. Kemper, J. Cui, K. Dicarantonio, K. Lin, D. Tang, A. Korikov, and S. Sanner. Retrieval-augmented conversational recommendation with prompt-based semi-structured natural language state tracking. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2786–2790, 2024.
- H.-D. Kieu, M.-D. Nguyen, T.-S. Nguyen, and D. D. Le. Keyword-driven retrieval-augmented large language models for cold-start user recommendations. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 2717–2721, 2025.
- S. Kim, G. Lee, K. Kim, J. Yoo, and K. Shin. Itemrag: Item-based retrieval-augmented generation for LLM-based recommendation. *CoRR*, abs/2511.15141, 2025.
- T. Kim, J. Kim, W.-Y. Shin, and S.-W. Kim. Leveraging retrieval-augmented language models for accurate item/feature selection in conversational recommender systems. In *Proceedings of the 19th ACM International Conference on Web Search and Data Mining, WSDM 2026*, 2026.
- C. Lao, F. Pan, G. Ma, H. Li, H. Lin, J. Shi, K. Zhao, K. Gai, M. Zhou, Q. Zhou, et al. Agentx: Towards agent-driven self-iteration of industrial recommender systems. 2026.
- C. P. Lee, J. Choi, and B. Mutlu. Map: Multi-user personalization with collaborative llm-powered agents. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, pages 1–11, 2025.
- B. Li, X. Wang, J. Li, W. Li, L. Zhang, S. Chen, W. X. Zhao, and J.-R. Wen. Recnet: Self-evolving preference propagation for agentic recommender systems. *arXiv preprint arXiv:2601.21609*, 2026a.
- C. Li, Z. Wang, H. Ma, W. Li, H. Wu, Y. Song, and M. Gao. Agentattack: Llm agents for multi-strategy shilling attacks in recommenders. *Expert Systems with Applications*, page 131163, 2026b.
- K. Li and K. H. Lim. Rallm-poi: Retrieval-augmented llm for zero-shot next poi recommendation with geographical reranking. In *AAAI*, pages 210–218. Springer, 2025.
- L. Li, Y. Zhang, D. Liu, and L. Chen. Large language models for generative recommendation: A survey and visionary discussions. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 10146–10159, 2024.
- Y. Li, X. Zhang, L. Luo, H. Chang, Y. Ren, I. King, and J. Li. G-refer: Graph retrieval-augmented large language model for explainable recommendation. In *Proceedings of the ACM on Web Conference 2025*, pages 240–251, 2025.

- J. Lian, Y. Lei, X. Huang, J. Yao, W. Xu, and X. Xie. Recai: Leveraging large language models for next-generation recommender systems. In *Companion Proceedings of the ACM Web Conference 2024*, pages 1031–1034, 2024.
- M. Liang, J. Wang, and Y. Luo. Smarteats: Investigating the effects of customizable conversational agent in dietary recommendations. In *Proceedings of the 7th ACM Conference on Conversational User Interfaces*, pages 1–16, 2025.
- J. M. Lichtenberg, A. Buchholz, and P. Schwöbel. Large language models as recommender systems: A study of popularity bias. *arXiv preprint arXiv:2406.01285*, 2024.
- Z. Lin, X. Zhang, K. Zhao, X. Lu, and Y. Zhang. Strajrag: Supervised trajectory retrieval augmented generation for next poi recommendation with travel semantics. *Information Processing & Management*, 62(6):104235, 2025.
- F. Liu, X. Lin, H. Yu, M. Wu, J. Wang, Q. Zhang, Z. Zhao, Y. Xia, Y. Zhang, W. Li, et al. Recoworld: Building simulated environments for agentic recommender systems. *arXiv preprint arXiv:2509.10397*, 2025a.
- H. Liu, Z. Sun, T. Wei, Y. Wang, J. Zhu, and X. Qu. Diagnostic-guided dynamic profile optimization for llm-based user simulators in sequential recommendation. In *AAAI*, 2026.
- J. Liu, S. Gu, D. Li, G. Zhang, M. Han, H. Gu, P. Zhang, T. Lu, L. Shang, and N. Gu. Agentcf++: Memory-enhanced llm-based agents for popularity-aware cross-domain recommendations. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2566–2571, 2025b.
- N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.
- J. Long, S. Huang, H. Huo, T. Chen, H. Yin, and G. Xu. Cloud-device collaborative agents for sequential recommendation. *arXiv:2509.01551*, 2025.
- Q. Long, H. Chen, C. Zhao, X. Du, X. Wang, P. Wang, C. Li, Y. Zhou, and H. Zhu. Sciencedb ai: An llm-driven agentic recommender system for large-scale scientific data sharing services. *arXiv preprint arXiv:2601.01118*, 2026.
- S. Luo, J. Xu, X. Zhang, L. Wang, S. Liu, H. Hou, and L. Song. Rallrec+: Retrieval augmented large language model recommendation with reasoning. *Expert Systems with Applications*, page 129508, 2025.
- B. Ma, H. Li, Z. Hu, X. Gui, L. Liu, and S. Lau. Agentrec: Next-generation llm-powered multi-agent collaborative recommendation with adaptive intelligence. *arXiv preprint arXiv:2510.01609*, 2025a.
- B. Ma, H. Li, Z. Hu, X. Gui, L. Liu, and S. Liu. Agenticrag: Tool-augmented foundation models for zero-shot explainable recommender systems. *arXiv preprint arXiv:2510.02668*, 2025b.
- R. Y. Maragheh and Y. Deldjoo. The future is agentic: Definitions, perspectives, and open challenges of multi-agent recommender systems. *arXiv:2507.02097*, 2025.
- R. Y. Maragheh, P. Vadla, P. Gupta, K. Zhao, A. Inan, K. Yao, J. Xu, P. Kanumala, J. Cho, and S. Kumar. Arag: Agentic retrieval augmented generation for personalized recommendation. *arXiv preprint arXiv:2506.21931*, 2025.

- C. Meng, H. Ling, J. Wang, Y. Liu, S. Zhang, D. Hong, M. Gao, O. Dalal, E. Chi, L. Hong, et al. Balancing fine-tuning and rag: A hybrid strategy for dynamic llm recommendation updates. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*, pages 919–922, 2025a.
- Z. Meng, Z. Yi, and I. Ounis. Kerag_r: Knowledge-enhanced retrieval-augmented generation for recommendation. *arXiv preprint arXiv:2507.05863*, 2025b.
- F. Nazary, Y. Deldjoo, T. Di Noia, and E. Di Sciascio. Stealthy llm-driven data poisoning attacks against embedding-based retrieval-augmented recommender systems. In *Adjunct Proceedings of the 33rd ACM Conference on User Modeling, Adaptation and Personalization*, pages 98–102, 2025.
- M.-D. Nguyen, H.-D. Kieu, and D. D. Le. Amem4rec: Leveraging cross-user similarity for memory evolution in agentic llm recommenders. *arXiv preprint arXiv:2602.08837*, 2026.
- G. Nie, R. Zhi, X. Yan, Y. Du, X. Zhang, J. Chen, M. Zhou, H. Chen, T. Li, Z. Cheng, et al. A hybrid multi-agent conversational recommender system with llm and search engine in e-commerce. In *Proceedings of the 18th ACM Conference on Recommender Systems*, pages 745–747, 2024.
- L. Ning, W. Fan, and Q. Li. Retrieval-augmented purifier for robust llm-empowered recommendation. *ACM Transactions on Information Systems*, 44(4):1–42, 2026.
- L.-b. Ning, S. Wang, W. Fan, Q. Li, X. Xu, H. Chen, and F. Huang. Cheatagent: Attacking llm-empowered recommender systems via llm agent. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2284–2295, 2024.
- K. Ordoumpozanis, G. Trichopoulos, and G. Caridakis. C-link agent. connecting social media post generation with recommender systems. In *2024 19th International Workshop on Semantic and Social Media Adaptation & Personalization (SMAP)*, pages 116–121. IEEE, 2024.
- C. Park. Llm as user simulator: Towards training news recommender without real user interactions. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 3080–3084, 2025.
- J. Park and M. Kim. Madrec: A multi-aspect driven llm agent for explainable and adaptive recommendation. *arXiv:2510.13371*, 2025.
- Q. Peng, H. Liu, H. Huang, Q. Yang, and M. Shao. A survey on llm-powered agents for recommender systems. *arXiv preprint arXiv:2502.10050*, 2025.
- I. D. S. Portugal, P. S. C. Alencar, and D. D. Cowan. An agentic ai-based multi-agent framework for recommender systems. In *2024 IEEE International Conference on Big Data (BigData)*, 2024.
- W. Qin, Y. Xu, W. Yu, C. Shen, X. Zhang, M. He, J. Fan, and J. Xu. More: A mixture of reflectors framework for large language model-based sequential recommendation. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems, RecSys '25*, page 299–308, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400713644. doi: 10.1145/3705328.3748055. URL <https://doi.org/10.1145/3705328.3748055>.
- Z. Qiu, L. Luo, Z. Zhao, S. Pan, and A. W.-C. Liew. Graph retrieval-augmented llm for conversational recommendation systems. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 344–355. Springer, 2025.
- J. Rao and J. Lin. Ramo: Retrieval-augmented generation for enhancing moocs recommendations. In *Proceedings of the HEXED/L3MNGET Workshop at EDM 2024*, 2024.

- K. Sayana, R. Vasudeva, Y. Vasilevski, K. Su, L. Hebert, J. Pine, H. Pham, A. Jash, and S. Sodhi. Beyond retrieval: Generating narratives in conversational recommender systems. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 2411–2420, 2025.
- T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- Y. Shang, P. Liu, Y. Yan, Z. Wu, L. Sheng, Y. Yu, C. Jiang, A. Zhang, F. Xu, Y. Wang, et al. Agentrekbench: Benchmarking llm agent-based personalized recommender systems. *arXiv preprint arXiv:2505.19623*, 2025.
- Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36:38154–38180, 2023.
- W. Shi, X. He, Y. Zhang, C. Gao, X. Li, J. Zhang, Q. Wang, and F. Feng. Large language models are learnable planners for long-term recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1893–1903, 2024a.
- W. Shi, X. He, Y. Zhang, C. Gao, X. Li, J. Zhang, Q. Wang, and F. Feng. Large language models are learnable planners for long-term recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1893–1903, 2024b.
- Y. Shi, H. Shang, X. Zi, W. Xu, Y. Feng, and M. Xu. Answering narrative-driven recommendation queries via a retrieve-rank paradigm and the OCG-agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025*, pages 13181–13202. Association for Computational Linguistics, 2025.
- Y. Shu, H. Zhang, H. Gu, P. Zhang, T. Lu, D. Li, and N. Gu. Rah! recsys–assistant–human: A human-centered recommendation framework with llm agents. *IEEE Transactions on Computational Social Systems*, 11(5):6759–6770, 2024.
- B. Škrlj, B. Guilleminot, and A. Tori. Agent0: Leveraging llm agents to discover multi-value features from text for enhanced recommendations. In *AI Agent for Information Retrieval Workshop*, pages 96–109. Springer, 2024.
- N. Sukiennik, H. Wang, Z. Zeng, C. Gao, and Y. Li. Simulating filter bubble on short-video recommender system with large language model agents. *arXiv preprint arXiv:2504.08742*, 2025.
- B. Sun, Y. Chen, J. Yang, X. Li, and H. Li. Retrieval-augmented recommendation explanation generation with hierarchical aggregation. *arXiv preprint arXiv:2507.09188*, 2025.
- S. Sun. CIRRR: Causal-invariant retrieval-augmented recommendation with faithful explanations under distribution shift. *CoRR*, abs/2512.18683, 2025.
- A. Tandon and A. Banerjee. Evaluating user intent classification and hybrid retrieval in a rag-based conversational tourism recommender system. 2025.
- J. Tang, Y. Luo, X. Xi, F. Sun, X. Feng, S. Dai, C. Yi, D. Chen, Z. Gao, Y. Li, et al. Interactive recommendation agent with active user commands. In *KDD*, 2026.
- A. Tourani, F. Nazary, and Y. Deldjoo. Rag-visualrec: An open resource for vision-and text-enhanced retrieval-augmented generation in recommendation. *arXiv preprint arXiv:2506.20817*, 2025.

- H. Wang, Y. Wu, D. Chang, L. Wei, and L. Heldt. Self-evolving recommendation system: End-to-end autonomous model optimization with llm agents. *arXiv preprint arXiv:2602.10226*, 2026a.
- J. Wang, B. Fu, Y. Zhu, and Y. Liu. Srlf: An agent-driven set-wise reflective learning framework for sequential recommendation, 2025a. URL <https://arxiv.org/abs/2511.11370>.
- L. Wang, J. Zhang, X. Chen, Y. Lin, R. Song, W. X. Zhao, and J.-R. Wen. Recagent: A novel simulation paradigm for recommender systems. *arXiv:2306.02552*, 2023a.
- L. Wang, J. Zhang, H. Yang, Z.-Y. Chen, J. Tang, Z. Zhang, X. Chen, Y. Lin, H. Sun, R. Song, X. Zhao, J. Xu, Z. Dou, J. Wang, and J.-R. Wen. User behavior simulation with large language model-based agents. *ACM Trans. Inf. Syst.*, 43(2), Jan. 2025b. ISSN 1046-8188. doi: 10.1145/3708985. URL <https://doi.org/10.1145/3708985>.
- L. Wang, J. Zhang, H. Yang, Z.-Y. Chen, J. Tang, Z. Zhang, X. Chen, Y. Lin, H. Sun, R. Song, et al. User behavior simulation with large language model-based agents. *ACM Transactions on Information Systems*, 43(2):1–37, 2025c.
- M. Wang, C. Gao, W. Wang, Y. Li, and F. Feng. Tunable llm-based proactive recommendation agent. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 19262–19276, 2025d.
- Q. Wang, D. Wang, K. Chen, Y. Hu, P. Girdhar, R. Wang, A. Gupta, C. Devella, W. Guo, S. Huang, et al. Adaptjobrec: Enhancing conversational career recommendation through an llm-powered agentic system. *arXiv preprint arXiv:2508.13423*, 2025e.
- S. Wang, W. Fan, Y. Feng, L. Shanru, X. Ma, S. Wang, and D. Yin. Knowledge graph retrieval-augmented generation for llm-based recommendation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27152–27168, 2025f.
- Y. Wang, Z. Liu, J. Zhang, W. Yao, S. Heinecke, and P. S. Yu. Drdt: Dynamic reflection with divergent thinking for llm-based sequential recommendation, 2023b. URL <https://arxiv.org/abs/2312.11336>.
- Y. Wang, Z. Jiang, Z. Chen, F. Yang, Y. Zhou, E. Cho, X. Fan, X. Huang, Y. Lu, and Y. Yang. Recmind: Large language model powered agent for recommendation, 2024a. URL <https://arxiv.org/abs/2308.14296>.
- Y. Wang, M. Li, L. Wang, K. Zhao, W. Li, W. Jiang, and X. Li. Momorec: A multi-agent motivation generation framework for residual semantic id-aware recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 15904–15914, 2026b.
- Z. Wang, Y. Du, Z. Sun, H. Chua, K. Feng, W. Wang, and J. Zhang. Re2llm: Reflective reinforcement large language model for session-based recommendation, 2024b. URL <https://arxiv.org/abs/2403.16427>.
- Z. Wang, Y. Yu, W. Zheng, W. Ma, and M. Zhang. Macrec: A multi-agent collaboration framework for recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '24*, page 2760–2764, New York, NY, USA, 2024c. Association for Computing Machinery. ISBN 9798400704314. doi: 10.1145/3626772.3657669. URL <https://doi.org/10.1145/3626772.3657669>.
- Z. Wang, Y. Yu, W. Zheng, W. Ma, and M. Zhang. Macrec: A multi-agent collaboration framework for recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2760–2764, 2024d.

- Z. Wang, M. Gao, J. Yu, Y. Hou, S. Sadiq, and H. Yin. Ruleagent: Discovering rules for recommendation denoising with autonomous language agents. *arXiv preprint arXiv:2503.23374*, 2025g.
- X. Wanyan, D. Hettiachchi, C. Ma, Z. Xu, and J. Chan. Temporal-aware user behaviour simulation with large language models for recommender systems. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pages 5335–5339, 2025.
- C. Wei, K. Duan, S. Zhuo, H. Wang, S. Huang, and J. Liu. Enhanced recommendation systems with retrieval-augmented large language model. *Journal of Artificial Intelligence Research*, 82:1147–1173, 2025a.
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- K. Wei, J. Gao, J. Zhong, Y. Yang, F. Lv, and Z. Li. Learning to shop like humans: A review-driven retrieval-augmented recommendation framework with llms. *arXiv preprint arXiv:2509.00698*, 2025b.
- T. Wei, H. Guo, Y. Du, Z. Sun, C. Huang, D. Wang, and J. Zhang. Mirroring users: Towards building preference-aligned user simulator with user feedback in recommendation. *arXiv preprint arXiv:2508.18142*, 2025c.
- C. Wu, R. Ren, J. Zhang, R. Wang, Z. Ma, Q. Ye, and W. X. Zhao. Starec: An efficient agent framework for recommender systems via autonomous deliberate reasoning. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pages 3355–3365, 2025a.
- J. Wu, C.-C. Chang, T. Yu, Z. He, J. Wang, Y. Hou, and J. McAuley. Coral: Collaborative retrieval-augmented large language models improve long-tail recommendation. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*, pages 3391–3401, 2024a.
- L. Wu, Z. Zheng, Z. Qiu, H. Wang, H. Gu, T. Shen, C. Qin, C. Zhu, H. Zhu, Q. Liu, et al. A survey on large language models for recommendation. *World Wide Web*, 27(5):60, 2024b.
- S. Wu, F. Han, B. Yao, T. Xie, X. Zhao, and D. Wang. Sunnie: An anthropomorphic llm-based conversational agent for mental well-being activity recommendation. *arXiv preprint arXiv:2405.13803*, 2024c.
- Y. Wu, Y. Peng, J. Yu, and R. Lee. Mas4poi: a multi-agents collaboration system for next poi recommendation. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 356–367. Springer, 2025b.
- Y. Wu, H. Wang, Q. Li, J. Zhang, H. Yu, and J. Jiang. Internalizing multi-agent reasoning for accurate and efficient llm-based recommendation. *arXiv preprint arXiv:2602.09829*, 2026.
- Z. Wu, L. Sheng, Y. Xia, Y. Zhang, Y. Chen, and A. Zhang. Personalized recommendation agents with self-consistency. In *Proceedings of the ACM Web Conference 2025*, 2025c.
- Y. Xia, S. Kim, T. Yu, R. A. Rossi, and J. McAuley. Multi-agent collaborative filtering: Orchestrating users and items for agentic recommendations. In *Proceedings of the ACM Web Conference 2026*, pages 8649–8652, 2026.

- Z. Xie, B. Peng, Z. He, Z. Chen, A. Han, I. Ye, B. Coleman, N. Sachdeva, F. Pereira, J. McAuley, W.-C. Kang, D. Z. Cheng, B. Wang, and R. Brown. Agentictagger: Structured item representation for recommendation with LLM agents. *CoRR*, abs/2602.05945, 2026.
- J. Xu, S. Luo, X. Chen, H. Huang, H. Hou, and L. Song. Rallrec: Improving retrieval augmented large language model recommendation with representation learning. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 1436–1440, 2025a.
- W. Xu, Y. Shi, Z. Liang, X. Ning, K. Mei, K. Wang, X. Zhu, M. Xu, and Y. Zhang. iagent: Llm agent as a shield between user and recommender systems. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 18056–18084, 2025b.
- M. Yang, N. Choudhary, J. Du, E. W. Huang, P. S. Yu, K. Subbian, and D. Kourta. Agentdr dynamic recommendation with implicit item-item relations via llm-based agents. *arXiv preprint arXiv:2510.05598*, 2025a.
- S. Yang, Z. Hu, C. Wang, T. Yu, X. Xu, L. Zhu, and L. Yao. Get the agents drunk: Memory perturbations in autonomous agent-based recommender systems. *arXiv e-prints*, pages arXiv–2503, 2025b.
- S. Yang, X. Li, G. Zhou, C. Wang, X. Xu, L. Zhu, and L. Yao. Retrieval-augmented review generation for poisoning recommender systems. *arXiv preprint arXiv:2508.15252*, 2025c.
- W. Yang, W. Zhang, Y. Liu, Y. Han, Y. Wang, J. Lee, and P. S. Yu. Cold-start recommendation with knowledge-guided retrieval-augmented generation. *arXiv e-prints*, pages arXiv–2505, 2025d.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36: 11809–11822, 2023.
- X. Ye, C. Xu, Z. Sun, J. Xu, G. Wang, Z. Dong, and J.-R. Wen. Llm-empowered creator simulation for long-term evaluation of recommender systems under information asymmetry. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 201–211, 2025.
- S.-e. Yoon, Z. He, J. Echterhoff, and J. McAuley. Evaluating large language models as generative user simulators for conversational recommendation. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1490–1504, 2024.
- H. Yu, Y. Wu, H. Wang, W. Guo, Y. Liu, Y. Li, Y. Ye, J. Du, and E. Chen. Thought-augmented planning for llm-powered interactive recommender agent. *arXiv preprint arXiv:2506.23485*, 2025a.
- Y. Yu, Z. Wang, C. Jiang, X. Li, J. Wang, and M. Zhang. Intelligent agents with adaptive knowledge fusion for personalized recommendation. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 2983–2987, 2025b.
- H. Zeng, Z. Yue, Q. Jiang, and D. Wang. Federated recommendation via hybrid retrieval augmented generation. In *2024 IEEE International Conference on Big Data (BigData)*, pages 8078–8087. IEEE, 2024a.

- Y. Zeng, A. Rajasekharan, P. Padalkar, K. Basu, J. Arias, and G. Gupta. Automated interactive domain-specific conversational agents that understand human dialogs. In *International Symposium on Practical Aspects of Declarative Languages*, pages 204–222. Springer, 2024b.
- A. Zhang, Y. Chen, L. Sheng, X. Wang, and T.-S. Chua. On generative agents in recommendation, 2024. URL <https://arxiv.org/abs/2310.10108>.
- G. Zhang, K. Zhao, J. Friedman, and X. Chu. Llms as orchestrators: Constraint-compliant multi-agent optimization for recommendation systems. *arXiv preprint arXiv:2601.19121*, 2026a.
- H. Zhang, C. Zhang, P. Hu, S. Qiu, and J. Wang. Customized retrieval-augmented generation with llm for debiasing recommendation unlearning. 2025a.
- J. Zhang, Y. Hou, R. Xie, W. Sun, J. McAuley, W. X. Zhao, L. Lin, and J.-R. Wen. Agentcf: Collaborative learning with autonomous language agents for recommender systems, 2023. URL <https://arxiv.org/abs/2310.09233>.
- S. Zhang, Y. Ma, J. Chen, S. Li, X. Yi, and H. Li. Towards aligning personalized conversational recommendation agents with users’ privacy preferences. *arXiv preprint arXiv:2508.07672*, 2025b.
- T. Zhang, N. Bisht, Z. Li, G. Xu, and X. Wang. Sarrec: Statistically-guaranteed augmented retrieval for recommendation. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management, CIKM 2025*, pages 4181–4190, 2025c.
- W. Zhang, X. Li, Q. Ge, K. Dong, P. Jia, X. Li, Z. Zhang, M. Wang, Y. Wang, H. Guo, et al. Exploring recommender system evaluation: A multi-modal llm agent framework for a/b testing. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pages 2878–2889, 2026b.
- Y. Zhang, S. Qiao, J. Zhang, T.-H. Lin, C. Gao, and Y. Li. A survey of large language model empowered agents for recommendation and search: Towards next-generation information retrieval. *arXiv preprint arXiv:2503.05659*, 2025d.
- Z. Zhang, S. Liu, Z. Liu, R. Zhong, Q. Cai, X. Zhao, C. Zhang, Q. Liu, and P. Jiang. Llm-powered user simulator for recommender system. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 13339–13347, 2025e.
- Y. Zhao, J. Wu, X. Wang, W. Tang, D. Wang, and M. De Rijke. Let me do it for you: Towards llm empowered recommendation via tool learning. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1796–1806, 2024a.
- Z. Zhao, W. Fan, J. Li, Y. Liu, X. Mei, Y. Wang, Z. Wen, F. Wang, X. Zhao, J. Tang, et al. Recommender systems in the era of large language models (llms). *IEEE Transactions on Knowledge and Data Engineering*, 36(11):6889–6907, 2024b.
- Z. Zhao, Y. Ding, W. Fan, and Q. Li. Webrec: Enhancing llm-based recommendations with attention-guided RAG from web. *CoRR*, abs/2511.14182, 2025.
- T. Zhe, R. Liu, F. Memar, X. Luo, W. Fan, X. Ye, Z. Peng, and D. Wang. Constraint-aware route recommendation from natural language via hierarchical llm agents. In *arXiv:2510.06078*, 2025.
- H. Zhong, H. Wang, Y. Ye, M. Zhang, and S. Zhu. Ggbond: Growing graph-based ai-agent society for socially-aware recommender simulation. *arXiv:2505.21154*, 2025.

- H. Zhou, H. Gu, Z. Zhan, X. Liu, K. Zhou, Y. Xiao, M. Liang, S. P. Govindan, P. Chawla, J. Yang, et al. The efficiency vs. accuracy trade-off: Optimizing rag-enhanced llm recommender systems using multi-head early exit. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 26443–26458, 2025.
- L. Zhu, X. Huang, and J. Sang. How reliable is your simulator? analysis on the limitations of current llm-based user simulators for conversational recommendation. In *Companion Proceedings of the ACM Web Conference 2024*, 2024.
- L. Zhu, X. Huang, and J. Sang. A llm-based controllable, scalable, human-involved user simulator framework for conversational recommender systems. In *Proceedings of the ACM on Web Conference 2025*, pages 4653–4661, 2025a.
- X. Zhu, Y. Wang, H. Gao, W. Xu, C. Wang, Z. Liu, K. Wang, M. Jin, L. Pang, Q. Weng, et al. Recommender systems meet large language model agents: A survey. *Foundations and Trends® in Privacy and Security*, 7(4):247–396, 2025b.
- Y. Zhu, H. Steck, D. Liang, Y. He, N. Kallus, and J. Li. Llm-based conversational recommendation agents with collaborative verbalized experience. Association for Computational Linguistics, 2025c.